

人間計算可能なパスワード認証に対する
埋め込みを前処理とする双方向長短期記憶
ニューラルネットワーク
による安全性評価

1TE20148Y 池田悠登

令和 6 年 2 月

九州大学 工学部 電気情報工学科

概要

人間計算可能なパスワードとは、ある秘密を人間が記憶し、それに基づいてチャレンジ（質問）に答えることで認証を可能にするものである。既存研究では、人間計算可能なパスワードのチャレンジ - レスポンスの組み合わせが漏洩したケースを想定し、数千 - 数万通りの組み合わせを機械学習を用いて学習し、認証に用いられる秘密の解読が可能であるかが研究されている。検証の結果として、ユーザーの記憶する画像枚数の多寡によらず、概ね 11% 程度しか推測できないことがわかっている。一方で、既存研究で試された機械学習モデルは多層パーセプトロンと長短期記憶のみであり、他にも様々な機械学習モデルが考えられるため、それらの攻撃への有効性の検証が課題であった。

本研究では、攻撃への有効性が高いと予測されるモデルを提案し、そのモデルを用いた攻撃を行っている。モデルとして、双方向長短期記憶と埋め込みを利用した。これは、これまで機械学習による推測が難しい要因とされていた非線形性を埋め込み層が補うことが可能であると考えたためである。埋め込み層は言語処理などの分野でよく活用される手法であり、入力した数値に対して対応するベクトルを出力することが可能である。数値 id に対応するベクトルを検索して出力することが可能であり、推測を難しくしていた非線形性・非連続性を解決するために有効な手段であると考えた。

本研究の結果として、ユーザーの記憶する画像枚数が 26 枚の場合は埋め込み層を用いて 50% 以上の精度で推測することが可能であった。また、ユーザーの記憶する画像枚数が 100 枚まで増えた場合でも 14% 程度までの正解率向上が確認された。一方で、ユーザーの記憶する画像枚数の増加に伴い正解率が低下することも確認された。これにより、人間計算可能なパスワードの安全性評価を行い、人間計算可能なパスワードを安全に運用するための条件を探ることができた。

Abstract

Commonly used passwords are strings of letters or numbers determined by humans. Password authentication using these passwords is widely used, but on the other hand, as the number of services used on a daily basis increases, the increased use of these passwords is causing security problems. On the other hand, it is complicated to use mechanically generated passwords with random numbers. Therefore, a human-computable password has been proposed.

A human-computable password is one that allows authentication by a human being storing a secret and answering a challenge to the question based on the secret. They are called human-computable because they do not require a large amount of memory or thinking and human can easily solve the problem. Existing studies have used machine learning to evaluate security. Assuming the case where a human-computable password challenge-response combination is compromised, thousands to tens of thousands of combinations are studied using machine learning to see if it is possible to crack the secret used for authentication. Existing research has tested the multilayer perceptron and long-short-term memory. It was difficult to guess difficult passwords. On the other hand, the limited number of machine learning models, which is mlp and lstm, have been tried in existing studies and we need to try better models.

In this study, I re-examined the security evaluation using a method called bidirectional long-short-term memory and embedding. This is because I guessed that the embedding layer can compensate for the nonlinearity, which has been considered a factor that makes inference by machine learning difficult.

As a result of this research, even human-computable passwords, which were previously considered difficult to analyze by machine learning, can be guessed with more than 50% accuracy using the embedding layer when user is trained with 24 images. In addition, even if user is trained with 100 images, it can be guessed about 14% accuracy.

目次

概要	i
Abstract	ii
図目次	v
表目次	vi
1 緒論	1
1.1 研究背景	1
1.2 本研究の貢献	2
1.3 本論文の貢献	2
2 背景知識	3
2.1 ユーザー認証の現状	3
2.1.1 認証とは	3
2.2 様々な方式の認証・認可	4
2.2.1 パスワード認証	4
2.2.2 チャレンジレスポンス認証	5
2.2.3 Passkey を用いた生体認証	6
2.2.4 Time-Based One Time password	7
2.3 一つのパスワードを安全に複数サービスで使う既存技術	8
3 機械学習	10
3.1 機械学習とは	10
3.1.1 機械学習で扱うデータの事前処理	11
3.1.2 モデルの構築方法	11
3.1.3 多層パーセプトロン	11
3.1.4 学習の実行	13
3.1.5 埋め込み層	14
3.1.6 LSTM	15
4 既存研究	18
4.1 人間計算可能なパスワード	18
4.1.1 人間計算可能なパスワードとは	18
4.1.2 人間計算可能の定義	19
4.1.3 一桁チャレンジ	21
4.2 CSPSolver による安全性検証	23
4.3 多層パーセプトロンによる安全性検証	24
4.4 長短期記憶による安全性検証	25
5 研究手法	27
5.1 本研究の情報学的な貢献	27

5.2	本研究の研究内容	27
5.3	なぜ埋め込み層を研究対象として設定したか	27
5.4	実験手法	29
6	結果	30
6.1	実験を行った環境	30
6.2	埋め込み層を用いた実験の結果	31
6.3	埋め込み層と長短期記憶を用いた実験の結果	38
6.4	考察	46
7	結論	48
	謝辞	49
	参考文献	50
	索引	52
	学会発表	52
	付録: プログラムコード	53

図目次

図 3.1.3.1 多層パーセプトロンの概要	12
図 3.1.5.1 埋め込み層の仕組み	14
図 3.1.6.1 LSTM の構成図	16
図 4.1.2.1 一桁チャレンジで暗記する画像の例	20
図 4.1.2.2 一桁チャレンジでの認証手順	
図 4.3.1 攻撃者による学習	22
図 4.3.2 学習データに基づくなりすまし	23
図 6.2.1 $N=26$, データ数 1000 MLP の実験結果	28
図 6.2.2 $N=26$, データ数 5000 MLP の実験結果	29
図 6.2.3 $N=26$, データ数 10000 MLP の実験結果	29
図 6.2.4 $N=26$, データ数 50000 MLP の実験結果	30
図 6.2.5 $N=50$, データ数 1000 MLP の実験結果	30
図 6.2.6 $N=50$, データ数 5000 MLP の実験結果	31
図 6.2.7 $N=50$, データ数 10000 MLP の実験結果	31
図 6.2.8 $N=50$, データ数 50000 MLP の実験結果	32
図 6.2.9 $N=100$, データ数 1000 MLP の実験結果	33
図 6.2.10 $N=100$, データ数 5000 MLP の実験結果	34
図 6.2.11 $N=100$, データ数 10000 MLP の実験結果	34
図 6.2.12 $N=100$, データ数 50000 MLP の実験結果	34
図 6.3.1 $N=26$, データ数 1000 LSTM の実験結果	35
図 6.3.2 $N=26$, データ数 5000 LSTM の実験結果	36
図 6.3.3 $N=26$, データ数 10000 LSTM の実験結果	36
図 6.3.4 $N=26$, データ数 50000 LSTM の実験結果	37
図 6.3.5 $N=50$, データ数 1000 LSTM の実験結果	37
図 6.3.6 $N=50$, データ数 5000 LSTM の実験結果	38
図 6.3.7 $N=50$, データ数 10000 LSTM の実験結果	38
図 6.3.8 $N=50$, データ数 50000 LSTM の実験結果	39
図 6.3.9 $N=100$, データ数 1000 LSTM の実験結果	39
図 6.3.10 $N=100$, データ数 5000 LSTM の実験結果	40
図 6.3.11 $N=100$, データ数 10000 LSTM の実験結果	40
図 6.3.12 $N=100$, データ数 50000 LSTM の実験結果	41

表目次

表 4.2.1 CSPSolver による解決にかかった時間	21
表 4.3.1 多層パーセプトロンによる攻撃の正解率	22
表 4.4.1 長短期記憶による攻撃の正解率	23
表 6.2.1 埋め込みと多層パーセプトロンの構成	27
表 6.2.2 埋め込みと多層パーセプトロンによる正解率	28
表 6.3.1 埋め込みと長短期記憶の構成	34
表 6.3.2 埋め込みと長短期記憶による攻撃の正解率	35

1 緒論

1.1 研究背景

現在様々な情報サービスが普及し、それらに対するユーザー認証の手法としてパスワード認証が広く使われている。パスワード認証は、ユーザーが使いたいサービスそれぞれに対して秘密の文字列を生成し認証のたびに入力することで行われる知識認証の一種である。ユーザーがもしコンピュータであれば、パスワード認証はサービスの数だけ乱数を生成しデータベースに記録することで安全に行うことが出来るが、実際の人間にはそのようなことは非常に難易度が高い。そのため、パスワードはしばしば使い回される。複数のサービスに対してパスワードが使い回されることで、1つのサービスからパスワードが漏洩すると他のサービスに対してもなりすましが行われてしまうこととなり、セキュリティ上の問題が存在する。これに対して提案されたものが、人間計算可能なパスワードである。

人間計算可能なパスワードは2016年にManuel Blumらによって提案された。このパスワードはチャレンジレスポンス方式となっており、人間はまず事前に画像と数値の対応付けと関数を記憶する。認証を行う際には、ユーザーは掲示された画像と記憶した秘密を元に関数の数値を計算し、入力する。ここで、認証を行うたびにユーザーに掲示される画像は別のものとなるため、認証を行うごとに入力されるパスワードは別のものとなる。掲示される画像の組み合わせの数が増えることによって、パスワードと違い多少の漏洩に対しても強固なセキュリティを実現できる。

既存研究では、ユーザーの閲覧した画像と入力した答えの組み合わせが流出したケースを想定し、多層パーセプトロン、RNNといった手法の機械学習による認証情報の推測が可能であるかが研究されている。結果として、提案されている人間計算可能なパスワードの手法は十分強固であり、機械学習による正解率はランダムに選択した場合の正解率とほとんど同じであることがわかっていく。

既存研究では、予測を難しくさせている原因として関数の非線形性が挙げられていた。そこで本研究では、非線形な関数を近似的に扱うことが出来る機構であるEmbeddingを用いて、これらの解析を行うことが可能であるかの検証を行っている。

1.2 本研究の貢献

提案当時、情報漏洩に十分強い認証方法だと考えられていた人間計算可能なパスワードに対して、機械学習を用いた安全性の検証を行った。チャレンジ – レスポンスの組み合わせが攻撃者に対して漏洩した場合を想定し、機械学習を用いてチャレンジ – レスポンスの組み合わせを模倣することで認証におけるなりすましがどの程度成立しうるかの調査を行った。結果として既存の人間計算可能なパスワードは安全性上問題があることが明らかになった。

また、現実的な攻撃者がなりすましを行う場合に、どの程度の情報漏洩が必要であるかの検証を行い、人間計算可能なパスワードを安全に活用する方法を模索した。

1.3 本論文の貢献

本論文ではまず認証に対する基礎知識として、パスワード認証やチャレンジレスポンス認証、OTP などのセキュリティ手法の説明を行う。

次に、既存研究の紹介を行う。まず、今回の安全性検証の対象となる手法である”人間計算可能なパスワード”の紹介を行う。また、多層パーセプトロンおよび RNN を用いて行われた既存の安全性検証についての結果を照会する。

そののち、人間計算可能なパスワードに対して、Embedding および双方向 LSTM を用いた機械学習モデルを活用した検証を行う。この際に、安全性に関わるパラメーターを複数変更し、人間計算可能なパスワードを安全に使える状況についての考察を行う。

2 背景知識

2.1 ユーザー認証の現状

研究背景として、始めに、ユーザー認証として使われている各種認証手法の現状について説明する。

2.1.1 認証とは

近年のモバイル端末の普及やデジタルサービスの多様化に伴い、個人が何らかのサービスなどに対して個人情報を伴う情報のやりとりを行う機会は増えている。これらのサービスはオンラインで非対面的に提供される場合がほとんどであるため、サービスの使用者の本人確認を行うための認証は欠かせない。対面で人を確認できない場合、通信内容が意図したものであるかどうか、通信している相手が本人であるかどうかを確認するためにユーザー認証が行われる。ユーザー認証にはパスワード認証や、FIDO アライアンスによって提案された Passkey を初めとするバイオメトリクスによる認証が使われている[23]。

認証に使われる手法は主に 3 つに大別される。本人しか知り得ない知識を用いて認証を行う知識認証、本人しか持っていない所有物を用いる所有物認証、ユーザーの生体情報を用いて認証する生体認証である[16][17][3]。

パスワード認証はこのうち知識認証に分類される。パスワード認証の仕組みについては詳しくは後述するが、ユーザーを特定する ID とそれを確認するための秘密であるパスワードを組み合わせることによってユーザー認証を行う。

生体認証には生物学的な特徴が用いられる。他人と異なる生物学的な特徴、例えば指紋や顔の相貌、虹彩や声紋などが用いられる。

所有物認証には所有物を用いる。例えば、鍵やキャッシュカード、後述するウェブサービスのワンタイムパスワードが例として挙げられる。

これらを比較する尺度として、「認証における安全性」と「ユーザーにとっての利便性」および「認証を行うために必要な設備のコスト」が存在する。

まず、認証における安全性で比較する。パスワードは使い回しなどにより破られるリスクが大きい。一方で、生体認証は方式によっては偽装難易度が高くなっている。例えば、虹彩認証や静脈認証は偽装が難しく、特に静脈認証は血流を用いて認証するため、認証媒体の盗難などに対しても耐性がある。そのため、本人以外を通してはいけないう高リスクな施設、例えばデータセンターなどでは静脈認証の活用例が多い。また、物理認証は盗難に弱いとため、鍵などの所有物を用いた認証は安全性が低いと言える。

次に利便性によって比較する。パスワード認証はパスワードの暗記が必要になる一方で、生体認

証、所有物認証ではユーザーは認証情報を暗記する必要がない。一方で所有物認証を行うためには所有物を持ち歩く必要がある。

最後にコストの観点から比較を行う。

パスワード認証はキーボード、マウスのみで認証できるため最も低コストである。生体認証、所有物認証は生体情報やカード情報などを読み取る設備が必要となり、コスト観点では不利であると言える。

近年は様々なセキュリティリスクの増加に伴い、複数種類の認証を組み合わせる本人確認を行う例が見られる。これらは多要素認証と呼ばれており、例えばログイン時にパスワードに加えてワンタイムパスワードを求めるウェブサービスが挙げられる。他にも、銀行のキャッシュカードも例として挙げられる。キャッシュカードを用いた所有物認証に加えて、自身で設定した暗証番号も用いて認証を行うことで、安全な預金の取引が可能となっている。

2.2 様々な方式の認証・認可

コストの観点認証の種類について前章で紹介した。本章では、様々な認証方式の概要とその事例の紹介を行う。

2.2.1 パスワード認証

パスワード認証は知識認証の一種である。サービス S に対するユーザー U のパスワードによる簡単な認証は次のような形式を取る。ただし、 $ID(X)$, $PASSWORD(X)$ はそれぞれ個人 X の ID, およびサービス S に対する個人 X の $PASSWORD$ を表す[3]。

$$U \rightarrow S: ID(S), password(S)$$

この方式で認証を行う場合、 $PASSWORD(S)$ はサービス S とユーザー U の間で事前に共有されている。この情報を SSL などを用いて暗号化した経路で送受信することで認証を行う手法が考えられる。

この手法の欠点としては、SSL による通信の暗号化が破られてしまった場合に、もう一度同じ通信を繰り返す手法であるリプレイ攻撃によってなりすまされることである。

2.2.2 チャレンジレスポンス認証

チャレンジレスポンス認証とは、一度認証に成功した通信をもう一度試みる攻撃であるリプレイ攻撃に対して耐性を持つ認証方式である。

この認証方式では、ユーザー – サービス間で認証情報を直接送受信せず、代わりに認証情報を持っていないと不可能な計算を行い、その結果をサーバーに送信することで認証情報が存在することを証明し認証を行う。

チャレンジレスポンス認証の最も簡単な例として、秘密鍵・公開鍵を用いた認証を挙げることができる[3][17]。

秘密鍵・公開鍵を用いた認証とは、楕円曲線暗号などを用いるチャレンジレスポンス認証の方法である。

この認証を行う際には、秘密鍵 SecretKey と公開鍵 PublicKey を用いる。ここで、これらの鍵を関数として使うことで次の計算が実行可能であるとする。なお、これらの計算をハッシュ計算と呼ぶ。

$$\text{SecretKey}(A) = B$$

$$\text{PublicKey}(B) = A$$

また、この関数の逆関数は計算量的に計算が難しいものとする。この場合に、次の手順を用いて認証を行うことが出来る。なお、具体的にこのような性質を満たすハッシュ関数として、SHA256 などが有名である。

1. ユーザーは自分のマシン上で秘密鍵と公開鍵のペアを作成する。
2. ユーザーは認証を行いたいサービスに事前に公開鍵を登録しておく。
3. 認証を行う前に、暗号化された安全な通信路をサービス・ユーザー間で確立する。
4. サービスはユーザーに対して、ランダムな文字列を送る（チャレンジ）
5. ユーザーはサービスから送られてきた文字列 A に対して SecretKey を用いて B への変換を行う。この行為を署名と呼ぶ。
6. ユーザーはサービスに対して、B を送る（レスポンス）
7. サービスはユーザーの公開鍵を用いて B を A への変換を行う。
8. B を A へ変換した結果が、サービスから送られてきた文字列 A と一致していれば認証成功とする。

このように、問題と答えのやりとりを行うことで、もし通信が漏洩しても認証におけるなりすましを行うことが困難となっている。これは、次に認証する際に使われる問題が別の問題であるた

め、前の問題と答えの組の情報があっても正しい答えを出せないことによるものである。

2.2.3 Passkey を用いた生体認証

Passkey とは、FIDO アライアンスによって提唱されたユーザー認証の方式である。[23]この方式はパスワードレスでの認証を目的として設計されており、顔認証・指紋認証などを用いて安全にウェブサイトに対して認証を行うことが出来る。さらに、顔認証の処理は端末側のみで行われるため、認証情報の流出の心配が無い。これは前述したチャレンジレスポンス方式での認証を活用しているためである。

ここで、文字列 S に対する秘密鍵 PrivKey での署名を $\text{Sign}(S, \text{PrivKey})$ と表す。また、署名済み文字列 S に対する公開鍵 PubKey での署名を $\text{Verify}(S, \text{PubKey})$ と表す。

Passkey による認証では、事前準備として次の処理が行われる。

1. ユーザーの所持する端末で秘密鍵・公開鍵ペアが生成される。この秘密鍵・公開鍵をそれぞれ PrivKey , PubKey とする。
2. サービスに対して公開鍵 PubKey が登録される。

認証フェーズでは次の手順で認証が行われる。

1. サービスからユーザーの端末に対して認証が要求される。このときにチャレンジである文字列 Str が送信される。
2. ユーザーの端末のソフトウェアがユーザーの生体情報を読み取る。
3. 生体情報が登録されたものと一致した場合、 $\text{Sign}(S, \text{PrivKey})$ が計算される。
4. 計算された $\text{Sign}(S, \text{PrivKey})$ がサービスに送信される。
5. サービスは $\text{Verify}(S, \text{PubKey})$ を計算し、 S と一致するかを確認する。

このような方法で、生体情報の送受信を伴わない生体認証が実現されている。一方で、本来生体認証を行わないとアクセスできない秘密鍵を攻撃者によって取り出されてしまうなどのリスクも指摘されている。

2.2.4 Time-Based One Time password

時刻ベースワンタイムパスワード(Time-Based One Time Password, TOTP)は、定期的に変更されるパスワードを用いた認証の方法である[24]。ここで、このワンタイムパスワードを表示することが可能な端末を限定することで、実際には所有物認証として扱われる。

この認証方式では、まず事前準備として、ユーザー - サーバー間で乱数から作られた文字列 Str を共有する。

ここで、ある時刻 t におけるワンタイムパスワード otp は次の関数から計算される。 otp は数分程度経過するごとに変化する数桁の文字列である。

$$otp = \text{Password}(t, \text{Str})$$

認証は次の手順で行われる。

1. ユーザーはコンピューターを用いて otp の値を計算する。
2. ユーザーはサービスに対して、ID と otp を送る。
3. サービスは現在時刻とユーザーID に紐付いた Str から otp を計算する。
4. otp の一致が確認された場合、認証が成功する。

ここで、この方式は一見 Str を秘密の知識とした知識認証の一種のように見える。しかし、実際の運用では、秘密 Str の保存と otp の計算をユーザーの特定の端末で行うため、設定された端末を所有していないと認証が行えない。そのため、所有物認証として扱われる。

この方式は知識認証では無く所有物認証のため、パスワード認証と組み合わせて多要素認証として実際に運用されている事例もある。

2.3 一つのパスワードを安全に複数サービスで使う既存技術

パスワードの使い回しを防ぐ方法として、次に挙げるような SAML や OpenID Connect, Active Directory といった手法が用いられる場合がある。これらはシングルサインオン(SSO)と呼ばれる手法であり、一つの認証情報を用いて一つのサービスに対して認証を行うことで、他のサービスへの認証も同時に済ませることが可能な手法となっている。また、これらのサービスは認証情報に加えてユーザーの属性情報も管理するため、その特徴から一般的には企業の情報システムでの利用例が多い。

従来の情報サービスでは、特定のサービスに対する認証情報はサービスに対して紐付いたものであり、同じ認証情報を複数のサービスで使い回すことはできなかった。SSO を用いることで、特定の一つの認証情報を安全に複数のサービスで活用することが可能となった。以下にそのような認証の例である SAML について説明する[16]。

Security Assertion Markup Language(SAML)とは、複数のウェブサイト・ドメイン間で認証情報の共有を行うために OASIS によって策定された技術規格である。SAML を用いることで、あるサービスに対する認証を別のサービスによって実施することが可能となる。SAML において、用語が次のように定義されている。

- Service Provider(SP)

ユーザーに対して実際にサービスを提供する情報システムである。例えば、SaaS 型のウェブサービスや企業内の情報システム、クラウドコンピューティングの管理コンソールなどが挙げられる。

- Identity Provider(IdP)

ユーザーの認証情報を管理するシステムである。例えば、Google 社が提供する企業向けの Google Workspace アカウントや、Okta 社が提供する認証サービス、Microsoft 社が提供する Azure Active Directory などがある。情報システムの管理者が自身でホストすることも可能である。

- ユーザー

実際に認証を行い、サービスの利用をするユーザーである。

SAML による認証フローには、サービス起点 (Service Provider Initiated) とユーザー起点 (Identity Provider Initiated) の 2 つのフローがある。このふたつは、ユーザー視点の体験とし

て最初に IdP のページへアクセスするか、SP のページへアクセスするかという違いがあるのみであり、認証フロー上特段して違う部分はない。ここでは、SP 起点のフローを例に説明する。

1. ユーザーが SP のページへアクセスする。
2. SP はユーザーの認証情報を IdP へリダイレクトする。
3. IdP はユーザーに対して認証を行うためのページを表示する。
4. ユーザーは IdP のページで認証を行う。
5. IdP は認証が成功した場合、SP へユーザーの認証情報をリダイレクトする。
6. SP はユーザーの認証情報を受け取り、ユーザーに対してサービスを提供する。

このようなフローを用いることで、SP 側でユーザーの認証情報を管理することなく、共通の認証基盤を用いて複数のサービスに対して認証を行うことが可能である。

3 機械学習

3.1 機械学習とは

機械学習とは、脳を模倣した構造であるニューラルネットワークと呼ばれる構造を用いてデータの処理・予測を行う手法である[5]。機械学習は複数の層とそれらの層のノードを接続する重みで構成される。機械学習の手法の例としては、多層パーセプトロン、RNN, CNN の他、大規模言語モデルで注目を浴びた Transformer などが代表的である[6]。これらモデルに対して特定のデータを学習させるために、損失関数と呼ばれる関数を用いてモデルの正しさの評価が行われる。損失関数で評価されるモデルの正しさを向上させるために、誤差逆伝播法と呼ばれる手法を用いてモデルに対して設定されるパラメーターの更新が行われる[7]。誤差逆伝播を用いて最適化する手法としては SGD, Adam, AdaGrad などが有名であり、これらは最適化アルゴリズムと呼ばれる[5]。

まとめると、機械学習では次のような手法で学習と推論が行われる。

1. データを学習させるためのモデルが用意される。また、モデルに対して学習を行うための、説明変数と正解データをセットとしたデータが大量に用意される。
2. モデルに対して重みがランダムに割り振られる。
3. モデルに対して学習データを入力し、損失関数によってモデルの正しさの評価が行われる。
4. モデルの正しさの評価を向上させるために、誤差逆伝播と最適化アルゴリズムを用いて重みが調整される。
5. 3, 4 の手順での学習を繰り返す。
6. 最後に、学習データとは別のデータを入力し、モデルが未知のデータにどれほど対応できるのか(汎化性能を持つか)を評価する。未知のデータにも対応出来るモデルでない場合は、学習データのみに対応するモデルとなり、この現象を過学習と呼ぶ。

過学習を起こすと、既存のデータのみにはしか対応出来ないモデルとなり、機械学習を用いる意味が無い状態になってしまう[5]。そのため、過学習は避ける必要がある。

機械学習への数値の入力と計算結果の出力はそれぞれベクトルデータや行列などのテンソルを用いて行われる。機械学習の最も入力側の層を入力層、最も出力側の層を出力層、それ以外の層を中間層とよぶ[5]。

機械学習のモデルは多層パーセプトロン、RNN, CNN や数値計算など多数の種類が存在する。

3.1.1 機械学習で扱うデータの事前処理

機械学習で扱うデータは、機械学習で扱いやすいように加工されて扱われる。例えば、巨大すぎる数値は 16bit や 32bit の浮動小数点値の範囲で扱いやすいように正規化され、画像の RGB 値も 3 チャンネルの別の値として扱われる場合が多い[5]。ここで、特徴的な One-Hot 表現について紹介する。

One-Hot 表現とは、データの分類をベクトルを用いて数値的に表すための手法である[4]。例えば、あるデータがそれぞれ「犬」「猫」「狸」のいずれかに分類される場合に、この分類の One-Hot 表現は(犬, 猫, 狸) = (1, 0, 0), (0, 1, 0), (0, 0, 1)のように表される。犬を 1, 猫を 2...のように数値に置き換えた場合には、機械学習の文脈ではある変数について犬より猫の方が大きく、狸はさらに大きいという意味合いになってしまう。このような意味合いを持たせずに学習と推論をおこなうため、One-Hot 表現が用いられる。

3.1.2 モデルの構築方法

一口に機械学習と言っても、様々なモデルを考えることができる。多層パーセプトロン、CNN, RNN などである[6]。ここで、先ほども説明したように、1 つの機械学習モデルは複数の層のつながりあわせとして表現される。以下に、研究対象とした層を中心に様々な層の説明を行う。なお、これらの層は自在に組み合わせることが可能である。例えば、RNN で使われる層と CNN で使われる層の両方を活用した機械学習モデルも存在する。

3.1.3 多層パーセプトロン

機械学習の層、および機械学習モデルの中で最も簡単なものが多層パーセプトロン(Multi Layer Perceptron, MLP)である[5]。ここでは多層パーセプトロンの計算を手順を追って説明する。多層パーセプトロンの構造は図 3.1.3.1 のようになっている。

ここで扱う多層パーセプトロンの中間層を n 層とし、入力データは i 次元ベクトル、出力データは o 次元ベクトルとする。各 k 層目に対して重み $W(k)$ とバイアス $b(k)$ が割り当てられているときに、 k 層目の出力を $x(k)$ とすると、 k 層目の出力は次のように計算される。

$$x(k) = f(x(k-1) * W(k) + b(k))$$

ここで、関数 f は活性化関数と呼ばれる関数である。活性化関数としては Sigmoid 関数や Rectified Linear Unit などの関数が一般的によく使用される。また、活性化関数として恒等関数を用いる場合もある。それぞれの活性化関数の計算式については後述する。

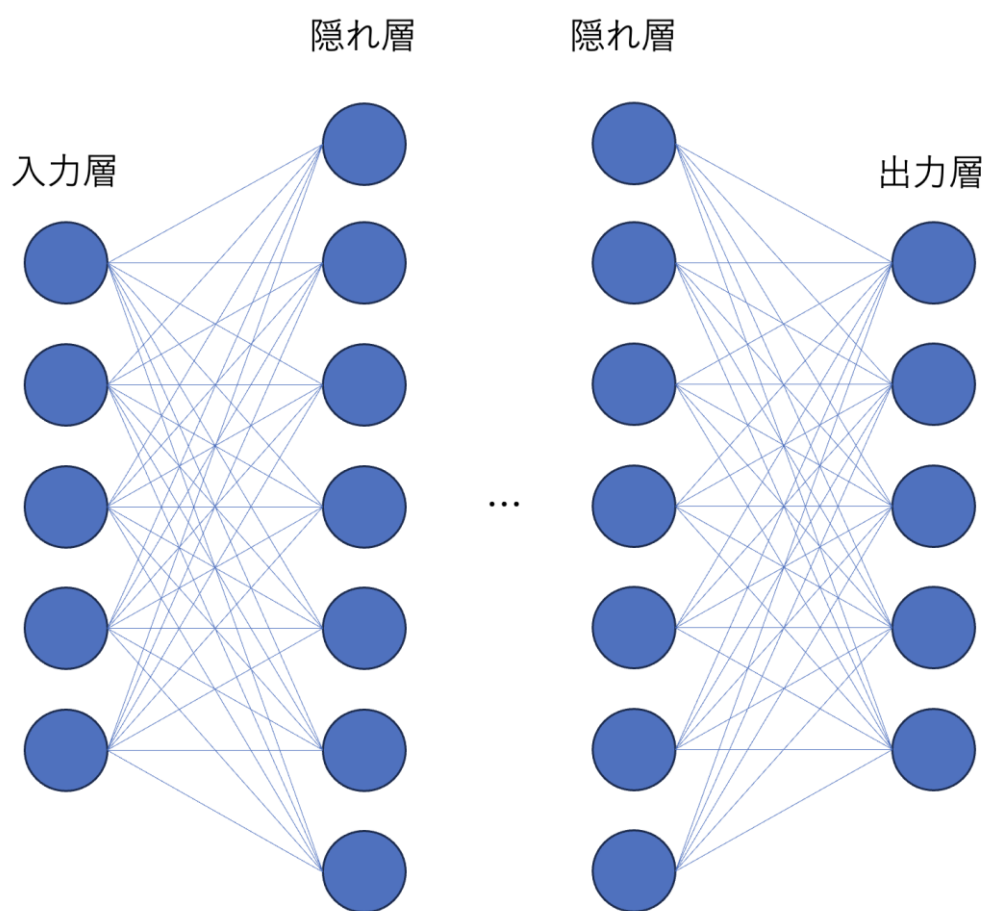


図 3.1.3.1 多層パーセプトロンの概要

3.1.4 学習の実行

機械学習は、事前にモデルを構築した上で、構築したモデルに大量のデータを学習させることで推論が可能となっている。ここでは多層パーセプトロンを例に実際にどのように学習が行われるかを紹介する。なお、ここで扱う方法は基本的には後述する他の層でも使われる手法である[5][7]。

機械学習の学習は、先ほども紹介したように次の手順を繰り返すことで実行される。

1. データを学習させるためのモデルが用意される。また、モデルに対して学習を行うための、説明変数と正解データをセットとしたデータが大量に用意される。
2. モデルに対して重みがランダムに割り振られる。
3. モデルに対して学習データを入力し、損失関数によってモデルの正しさの評価が行われる。
4. モデルの正しさの評価を向上させるために、誤差逆伝搬と最適化アルゴリズムを用いて重みが調整される。
5. 3, 4 の手順での学習を繰り返す。

ここでは、逆誤差伝搬法と最適化アルゴリズムについて注目して説明を行う。

上で、損失関数と呼ばれる関数を用いることで機械学習が行っている推論の結果と実際の結果の誤差を表現できることを紹介した。逆誤差伝搬法は、この誤差の結果を参考に機械学習上のパラメータをどのように変更すればよいのか探索するための手法である。

誤差逆伝搬法では、各レイヤーのパラメータに対して損失関数の値に対する微分値を計算する。微分を計算することで、損失関数の値を低下させて精度を向上させるためにそのパラメータを増加させるべきなのか減少させるべきなのかを判断することが可能である。これに対して最適化アルゴリズムを用いることで、実際にそのパラメータをどの程度変化させればよいかを決定し、実際に決定を行う。また、最適化アルゴリズムに対して設定される、一度にどの程度を目安にパラメータ値を変化させるかを指示する数値を学習率と呼ぶ。

ここで、最適化アルゴリズムが参考にするのは、現在設定されているパラメータに対する偏微分値である。最適化を一度行った後の損失関数の値が低下していることの確認を行っていない。そのため、機械学習は、長期的に学習を行っている場合は結果が改善している場合でも、一回の学習だけでは損失関数の値が悪化する方向に数値が変化する可能性がある。例えば、学習率が大きすぎる場合には、損失関数が小さくなる部分がグラフ上谷の形となっており、学習した結果行き過ぎてしまい谷より先の山の部分に設定される場合がある。このような性質により、機械学習の学習中の損失関数の値をグラフにて観察すると概ねギザギザした形になるが、これは使っているアルゴリズム上の特性であり、バグなどを疑う必要はない。

3.1.5 埋め込み層

埋め込み層とは、単語の分散表現と呼ばれる手法でコンピュータ上で効率的に自然言語処理を行うために提案された層である[6]。この層の概略図を図 3.1.5.1 に示す。

ここで、単語の分散表現について説明する。

単語の分散表現とは、単語の意味をベクトルを用いて表現する手法である[6]。かんたんに言えば、ベクトルは次のような性質を満たす。

- ベクトル同士の足し算・引き算が可能である。
- ベクトル同士の内積が可能である。
- ベクトル同士の距離を計算することが可能である。

ここで、ベクトル同士の内積や距離は単語同士の意味の近さを表すことができる。また、単語の足し算・引き算によって単語の意味合いの関係を表すことが可能である。

この層は、入力として分散表現の id を受け取り、出力として対応する id のベクトルを出力する。ここで、Embedding は近い id に対しても飛び飛びの値を返すことが可能である。

埋め込み層の構造は図 3.1.5.1 のようになっている。数値の入力に対して、まずこの入力が One-Hot ベクトルに変換される。そののち One-Hot ベクトルと行列のかけ算が計算され、行列の特定の列が選択され出力される。

埋め込み層を実際の問題解決に活用した例は様々である。例えば、自然言語処理と呼ばれる分野でコンピュータに人間の言葉を理解させて情報処理を行う手法には一般的に採用されている。活用例として、単語のベクトルを用いて文章を意味合いから検索できるベクトル類似性検索がある。

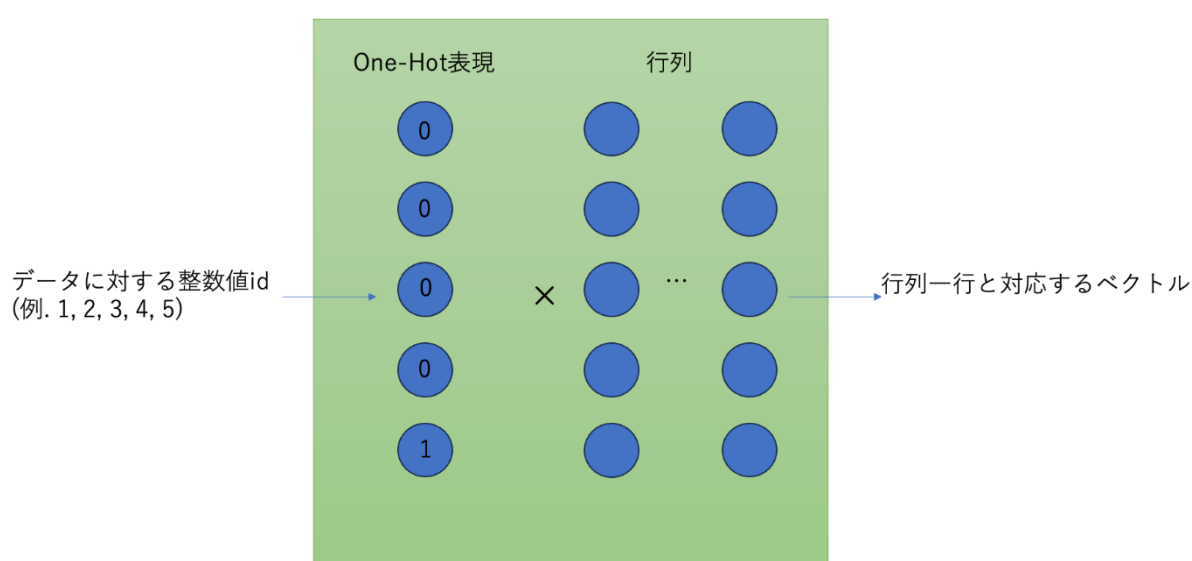


図 3.1.5.1 埋め込み層の仕組み

3.1.6 LSTM

Long Short Term Memory(LSTM)とは、長短期記憶とも呼ばれる手法である。この手法の概略図は図 3.1.5.1 にある。この手法では、時系列データを順にモデルに対して入力し、時系列データに対する推論を出力する[5] [6] [7]。 よって特徴として、時系列データの処理に向いているという点が挙げられる。

時系列に対する予測がなぜ可能なのか？という質問に対して答えると、これは LSTM が内部に状態を持つからである。図 3.1.6.1 に示されるように、LSTM は内部に状態を持つ。時刻 t における入力を $x(t)$ 、出力を $y(t)$ 、内部の状態を $h(t)$ とすると、RNN の計算は次のように表される。

$$h(t) = f(x(t), h(t-1))$$

$$y(t) = g(h(t))$$

LSTM はこのような手法で内部に状態を持つことで、任意の長さの時系列データに対する処理を実現している。

なお、一般的な LSTM では時系列に進む方向にしか学習しない。だが、動画の補完処理など逆方向にも学習・推論を行いたいケースがある。そのような場合は、両方向に学習を行う手法である双方向 LSTM が広く活用されている。

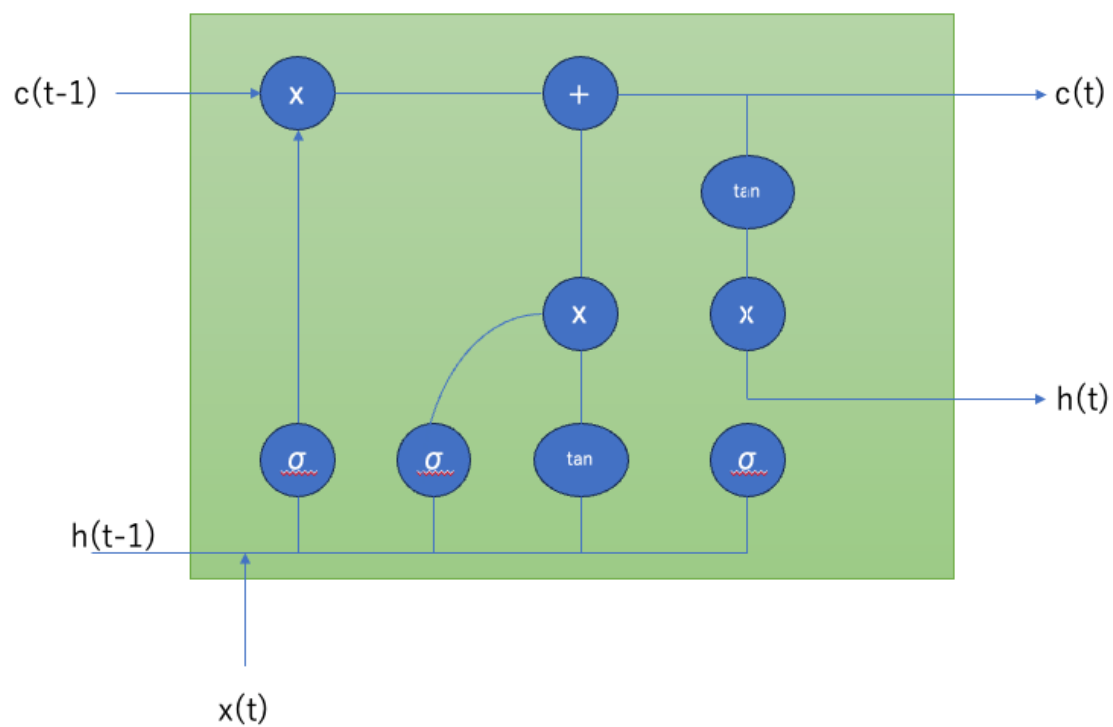


図 3.1.6.1 LSTM の構成図

3.1.7 運任せな部分もある機械学習

推論を機械学習モデルを用いて行うために、まず各パラメーターを乱数で設定したモデルを準備して、そのモデルを学習させることで正解率を向上させることは前節までで説明した。

実は、機械学習は同じモデルを作成して訓練するプログラムであっても実行するごとに結果が変わる。これは、モデルに最初に設定されているパラメーターが乱数で決定されるためである[5]。

乱数によって最終的な機械学習の性能が変化してしまう大きな原因の一つに、局所解のトラップと呼ばれる現象がある。これは、一定距離以内のパラメーターの中で最も損失関数が低い箇所にパラメーターが固定されてしまう現象である。多くの最適化アルゴリズムでは、学習率が一定であるか、学習が進むにつれて精度高くパラメーターを変更するために学習率が低くなる。ここで、数学的に十分遠いパラメーターは探索されないため、局所的な解にパラメーターが固定されてしまう。これによって、本来例えば正解率が80%のモデルでも実行ごとに70%や30%といった結果が出ることもある。

3.1.8 普遍近似定理

ニューラルネットワークに関する興味深い性質として、普遍近似定理が知られている。これは、任意の連続な関数について、3層のニューラルネットワークを用いることで表現できるというものである[25] [3]。この性質から、ニューラルネットワークは万能の近似器と呼ばれることもある。

4 既存研究

4.1 人間計算可能なパスワード

4.1.1 人間計算可能なパスワードとは

人間計算可能なパスワードとは、2016 年に Manuel Blum らによって提案された認証の手法である。この手法の特徴は、ユーザーのコンピューターを用いずに脳内で計算が完結するチャレンジ – レスポンス認証であることである。人間の脳内で計算が完結することから人間計算可能と呼ばれ、知識認証に分類されることからパスワードと呼ばれている[1]。人間計算可能の定義および実際の認証の方法については次節以降で紹介する。

人間計算可能なパスワードは、大きく分けて PRPE と PROC の二段階に分けて整理される[1][2]。PRPE は実際のシステムにパスワードを登録して利用するよりも前の段階であり、パスワードの生成・記憶が該当する。それに対して PROC は実際に鍵と入力に対して処理を行い認証を行うことが該当する。さらに、準備段階である PRPE は MEM と COM に分類される。MEM は秘密鍵自体を人間が生成し記憶することであり、これには記憶を保持するためにリハーサルを行うことも含む。また COMM は計算のスキーマ自体の学習であり、鍵と入力に対してどのような演算を行うかの学習が該当する。

4.1.2 人間計算可能の定義

人間計算可能であることの定義を行うために、まず人間を次のようにモデル化して、一種の計算機だと見なす[2]。

1. 人間は一種のチューリングマシンであるが、通常のチューリングマシンと違いメモリが長期用・短期用の2つに分かれている。
2. 長期記憶の記憶容量は無限大である。読み出しは素早く行えるが、書き込みには一定の時間を必要とする。
3. 短期記憶は書き込み・読みだしともに光速である。ただし容量に制限があり、通常 2-3 個しか記憶を行えない。

この仮定に基づき、前節で整理した PRPE および PROC が人間計算可能であるための条件は次のように定義されている。なお、ここでは要する時間が 10 分以内であるなどの具体的な制約があるが、これは Manuel Blum ら論文の筆者らが現実的にユーザーが要することができる時間として設定した定数である。

1. COMM に要する時間: COMM ではスキーマの学習を行う。この時間には、計算

の方法の学習とリハーサルが該当する。これは 10 分以内に完結するものである必要がある。

2. PRPE に要する時間: COMM と MEM を合わせた、ユーザーがパスワード設定前の準備に要する時間である。これは二時間以内に完結するものである必要がある。
3. PROC に要する時間: 実際に認証を行う際に、1 つの入力に対して入力と秘密鍵からレスポンスを生成するための時間である。一分以内である必要がある。
4. スキーマが最大 3 つであり、可能な限り 2 つのチャンクは記憶に対するポインタとして利用される。
5. スキーマおよび鍵を記憶した後、自分自身でリハーサルを行い確認することが可能である。

これらの制限は、現実的なユーザーが外部の記憶媒体(PC や計算用紙など)を用いずに認証を行うために設定されたものである。認証に要する時間が長すぎる場合は利用が現実的ではなくなり、脳内で完結しない計算である場合は計算に使用した PC や紙媒体などに秘密を含む情報が残されてしまうことになる。これらの事態を防ぐために、このような制約が定義されている。

4.1.3 一桁チャレンジ

一桁チャレンジとは人間計算可能なパスワードの一種である。問題と鍵から計算される答えが 0-9 の一桁の数値であることからこのように呼ばれる。このパスワードの特徴として、画像と数値の組み合わせを計算することにある。

以下に、事前準備と実際の認証の二段階に分けて本方式を紹介する。

1. 事前準備

ユーザーは事前知識として N 枚の画像と数値の組み合わせ、および計算に用いる関数を暗記する。具体的には、ある画像を見たユーザーはその画像に紐付く数値を思い出せる必要がある。これをマッピング関数 σ と呼ぶ。例を図 4.1.2.1 に示す。

さらに、ユーザーは数値を入力として出力を計算する関数 f を記憶する。

まとめると、 σ 関数は画像を入力として受け取り、0-9 の整数に変換する関数である。それに加えて、関数 f を定義する。この関数 f は、次の式で定義される。

2. 認証

認証を行う際には、ユーザーが登録した 100 枚以内の画像の中からランダムに 14 枚の画像がユーザーに掲示される。この画像をもとに、ユーザーは計算を行い認証する。

この計算によって得られた計算結果がユーザーが認証に用いる一桁の認証情報である。

認証情報を数式で表すと次のようになる[1], [2]。

$$f(x_{\{0\}}, x_{\{1\}}, \dots, x_{\{13\}}) = x_j + x_{\{12\}} + x_{\{13\}} \bmod 10$$

$$j = (x_{\{10\}} + x_{\{11\}}) \bmod 10$$

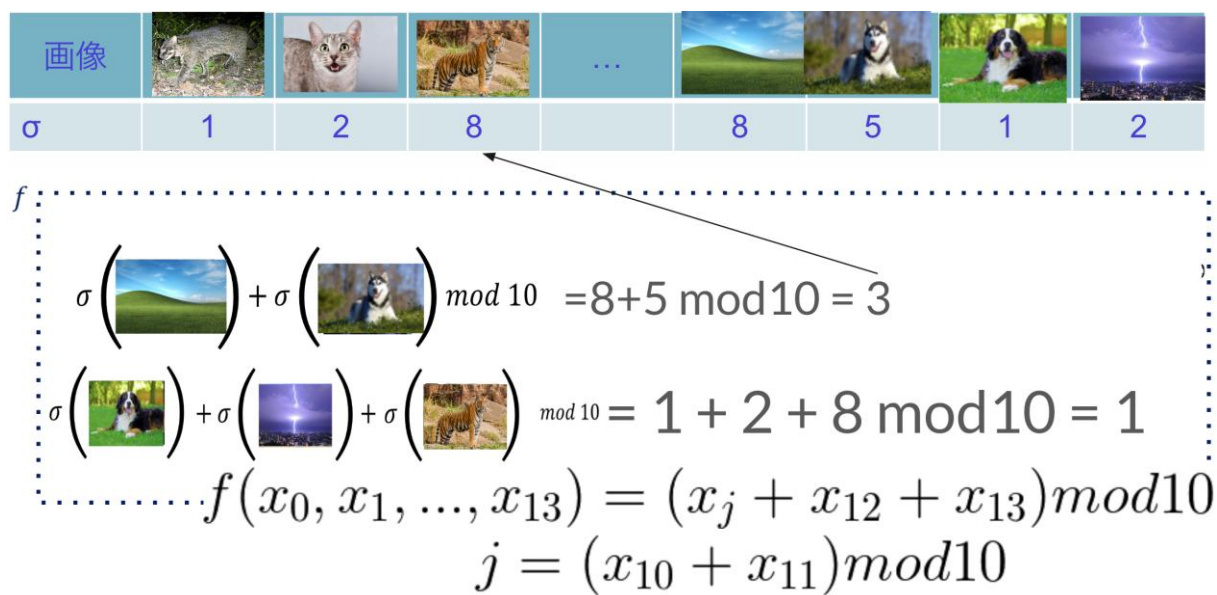


図 4.1.2.1 一桁チャレンジで暗記する画像と計算



図 4.1.2.2 一桁チャレンジでの認証手順

4.2 CSPSolver による安全性検証

既存研究では、制約充足性問題(Constraint Satisfaction Problem)としてユーザーの秘密を求めることができるかの検証が行われている[1] [2]。実験は次のような手順で行われた。

CSPSolver は一桁チャレンジにてユーザーに掲示された 14 枚の画像と関数を受け取り、制約を満たす秘密の探索を行う。ここで、Solver が計算開始から 2.5 日以内に秘密を答えることができれば失敗したと見なしている。なお、計算には 2.83GHz Intel Core2Quad と 4GB RAM を搭載したコンピューターが使われている。

ユーザーが覚える画像の枚数を n , Solver が受け取るデータの数 m としたときに、表 4.2.1 のような結果となっている。

	$m=50$	$m=100$	$m=300$	$m=500$	$m=1000$	$m=10000$
$n=26$	23.5h	40min	4.5h	29min	10min	2min
$n=30$	解決不可	解決不可	2.33h	35.5min	10min	20sec
$n=50$	解決不可	解決不可	解決不可	解決不可	解決不可	7h
$n=100$	解決不可	解決不可	解決不可	解決不可	解決不可	解決不可

表 4.2.1 CSPSolver による解決にかかった時間

これらのことから、ユーザーが暗記する画像枚数を 100 枚以上とした場合は、仮に関数が既知のものであっても解析することが出来ないことがわかる。

4.3 多層パーセプトロンによる安全性検証

先行研究では、多層パーセプトロンを用いた解析の実験も行われている[3]。

この研究で想定されている状況の概略が図 4.3.1, 図 4.3.2 である。

まず、実験内容について説明する。この実験では、図 4.3.1 のように通信路上などからユーザーの閲覧した 14 枚の画像の組み合わせとユーザーの入力した数値が漏洩した場合を想定している。多層パーセプトロンのモデルに対して、ユーザーの閲覧した 14 枚の画像を説明変数、ユーザーの入力した一桁の数値を目的変数として学習を行わせる。学習を行ったモデルが、図 4.3.2 のようにユーザーになりすますことに成功するかどうかを評価する。

実験を行った結果として、表 4.3.1 のような結果が得られている。ここで、ランダムに答えを選択した場合でも正解率は 10%となるため、予測に成功しているとは言えない。

データ数	中間層の数	中間層のノード数	予測精度 n=26	予測精度 n=30	予測精度 n=50	予測精度 n=100
1000	3	128,64,32	0.0800	0.0975	0.1033	0.0800
1000	4	128,64,32,16	0.0900	0.0850	0.1133	0.0950
1000	4	256,128,64,32	0.1067	0.0850	0.1033	0.0800
10000	3	128,64,32	0.1017	0.1050	0.0980	0.1060
10000	4	128,64,32,16	0.0987	0.0860	0.0890	0.0905
10000	5	256,128,64,32,16	0.0980	0.0915	0.1017	0.1010
100000	4	128,64,32,16	0.1174	0.1013	0.1012	0.0991
100000	5	256,128,64,32,16	0.1045	0.1041	0.0996	0.0976
100000	6	512,256,128,64,32,16	0.1022	0.1032	0.0990	0.1007

表 4.3.1 多層パーセプトロンによる攻撃の正解率

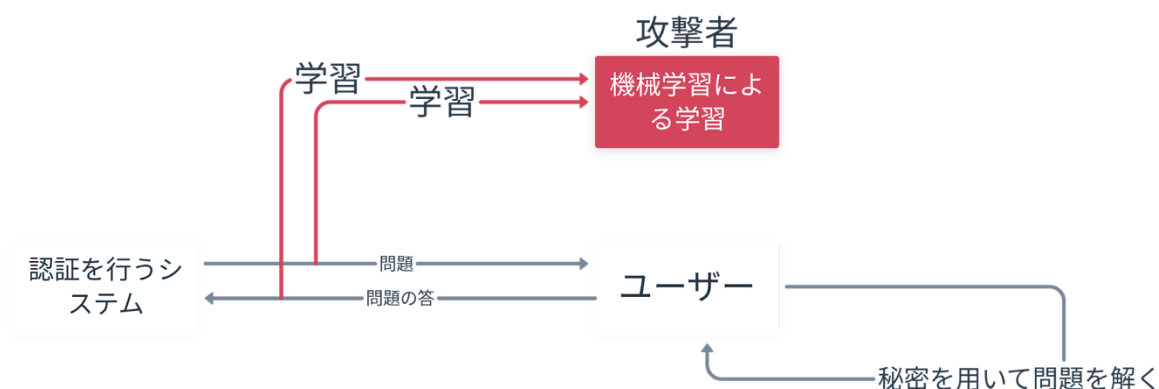


図 4.3.1 攻撃者による学習

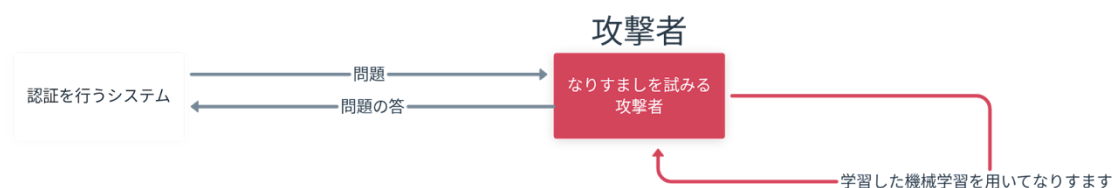


図 4.3.2 学習データに基づくなりすまし

また、先行研究では、予測対象を変更したまま、次の2つのケースを機械学習で予測可能であるかも実験が行われている。

- ユーザーの秘密であるマッピング関数がわかっている場合(関数を予測する)
- マッピング関数が既知である上に、事前にどの値を加算して計算するかわかっている場合

結果として、ユーザーの秘密がわかっている場合でも関数の予測を行うことが難しいという結論が出ている。また、マッピング関数が既知である上でさらにどの値を加算するかがわかっている場合は86%程度の正解率での予測が可能であるという結論も出ている。これらのことから、機械学習による予測を難しくしている要素として、関数が大きな役割を果たしていることが推測できる。

4.4 長短期記憶による安全性検証

多層パーセプトロンと同様の設定で、長短期記憶モデルを用いた安全性評価も先行研究で行われている[4]。この実験の内容は上述した多層パーセプトロンのものとほぼ同様であるが、実験されている条件が少ないことと、モデルが長短期記憶に変更されていることが特徴である。この実験の結果は表 4.4.1 のようになっている。

データ数	画像枚数 26	画像枚数 50	画像枚数 100
10000	0.1002	0.0979	0.0973
50000	0.1121	0.1004	0.0989
100000	0.1152	0.1023	0.1008

表 4.4.1 長短期記憶による攻撃の正解率

この結果から、長短期記憶を用いて人間計算可能なパスワードの推測を行うことは不可能である

と判断することができる。

また、既存研究では、多層パーセプトロンの際同様に、関数自体が予測可能であるか、およびどの値を加算して計算するかわかっている場合についても評価を行っている。

結果として、関数自体の予測は 15%程度しか正解していない。また、どの値を加算するかわかっている場合は正解率は 98%程度となっており、ほとんど正確に予測することに成功している。

5 研究手法

5.1 本研究の情報学的な貢献

本研究の情報学的な貢献としては、二つのものを考えている。一つは、人間計算可能なパスワード自体に対する安全性の検証を行い、安全に利用できる限度を示すことでより強固な情報化社会の実現に役立てることである。もう一つは、益々高度化する機械学習を用いてセキュリティの検証を行い、攻撃および安全性評価における機械学習の有用性を示すことである。

5.2 本研究の研究内容

本研究では、既存研究同様に機械学習を用いて人間計算可能なパスワードの予測が可能であるかの検証を行う。この際に、双方向化した長短期記憶モデルおよび埋め込み層を用いたモデルを用いて研究を行う。本研究のきっかけは、人間計算可能なパスワードにおける σ 関数の計算内容を埋め込み層を用いて再現できるのではと考えたことによる。さらに、データ数および画像数を複数変えて実験を行い、どのような状況であれば安全に人間計算可能なパスワードを活用することが可能であるかの評価も行う。

5.3 なぜ埋め込み層を研究対象として設定したか

既存研究では、人間計算可能なパスワードを機械学習にて推測する場合に難易度が高くなる原因として、 σ 関数の存在および関数の非線形性が指摘されていた。掲示される画像によって3枚目に参照する画像が変わることや、そもそも掲示された画像に便宜的に振ったidと実際に計算に使われる数値が違うことが推測を難しくしていると予測されてきた。

そこで、埋め込み層を活用してこの点の緩和を行うことを考えた。埋め込み層では、単語idに対してその単語に対応するベクトル表現を抽出することが可能である。この仕組みを画像id- σ 関数の対応付けとして使うことが可能であると考えたことが本研究に埋め込み層を使うことに決めたきっかけである。

5.4 実験手法

本研究の実験は、次の手順で行われた。まず、乱数を用いて一桁チャレンジの秘密である関数 σ をランダムに作成する。次に、 σ に基づきチャレンジ - レスポンスの組み合わせをランダムに生成する。この際に生成されたデータの数、ユーザーから漏洩した認証データの数と同様であると考えることができる。生成されたチャレンジ - レスポンスの組み合わせを 9:1 の割合で学習データと検証データに分割し、学習データを機械学習によって学習する。学習を行ったモデルでの検証データの正解率を観察し、この正解率を人間計算可能なパスワードの正解率として実験を行う。

なお、機械学習モデルの構築には Google 社が開発した Keras というフレームワークを用いている[26][27]。Keras は同社が開発した機械学習フレームワークである tensorflow を抽象化して扱うことが可能なラッパーライブラリである[26][28]。ただし、データに対する次元の操作などに非対応のため、一部 tensorflow の関数を直接使用した[29]。

6 結果

6.1 実験を行った環境

本実験を行ったマシンのハードウェア環境は次の通りである。

CPU: Intel Core i7-12700KF

RAM: 128GB(swap 領域設定なし)

GPU: nVidia GA102[RTX 3080Ti 12GB]

また、次のバージョンのソフトウェアを用いてコンピュータを稼働させた。

OS: Ubuntu 22.04 LTS

機械学習プログラムの動作に用いた各種ライブラリのバージョンは次の通りである。

Conda: 23.7.0

Python: 3.11

Tensorflow: 2.15.0

Keras: Tensorflow 2.15.0 に付属のバージョン

CUDA: 12.2

cuDNN: 8.9

6.2 埋め込み層を用いた実験の結果

ユーザーが覚える必要がある画像の枚数 n について、 $n=26, 30, 50, 100$ に対して実験を行った。ここで、実験に用いた機械学習モデルは表 6.2.1 のようなものである。

層 id	層	ノード数	出力形式(行列やテンソルの形式)
1	入力層		(14)
2	埋め込み層	14(語彙数)	(14, 3)
3	連結(1, 2)		(14, 4)
4	パーセプトロン	50	(50)
5	パーセプトロン	50	(50)
6	パーセプトロン	50	(50)
7	パーセプトロン(出力層)	10	(10)

表 6.2.1 埋め込みと多層パーセプトロンの構成

このモデルを訓練した結果、次の表 6.2.2 の結果となった。

データ数	N=26	N=50	N=100
1000	13.50%	11.90%	13.20%
5000	11.22%	12.16%	12.98%
10000	18.52%	11.14%	11.21%
50000	27.1%	11.35%	12.34%

表 6.2.2 埋め込みと多層パーセプトロンによる正解率

また学習中のグラフは、それぞれ次の図 6.1.1 から 6.1.12 のようになった。

全体的に学習の初期段階から過学習が起こっている傾向が見受けられる。また、グラフの振れ幅が全体的に激しく、学習率が高すぎる可能性もあると考えられる。

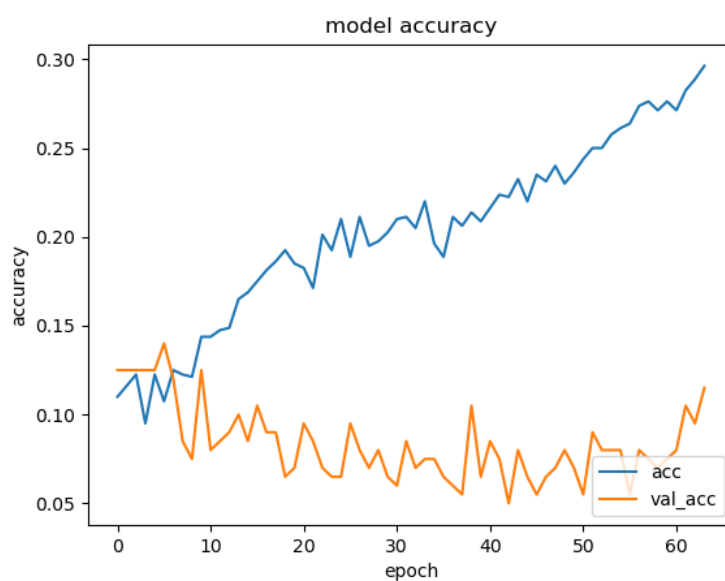


図 6.2.1 N=26, データ数 1000 MLP の実験結果

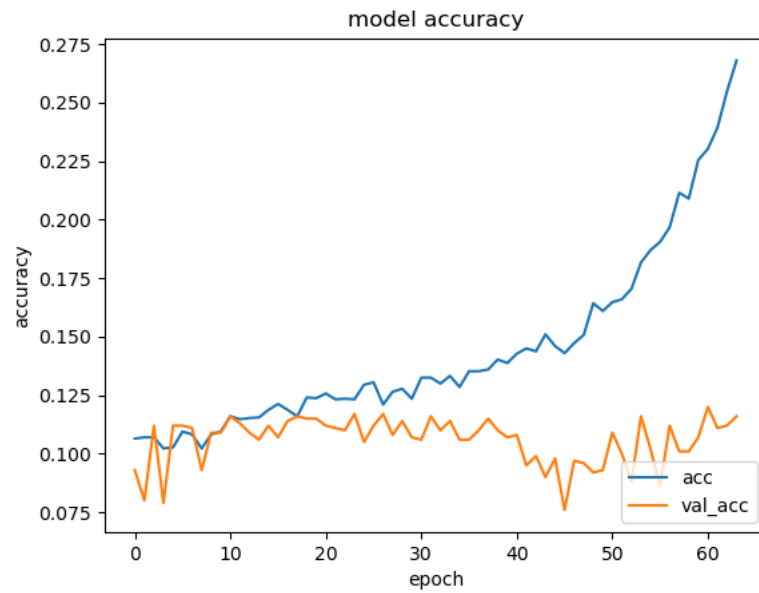


図 6.2.2 $N=26$, データ数 5000 MLP の実験結果

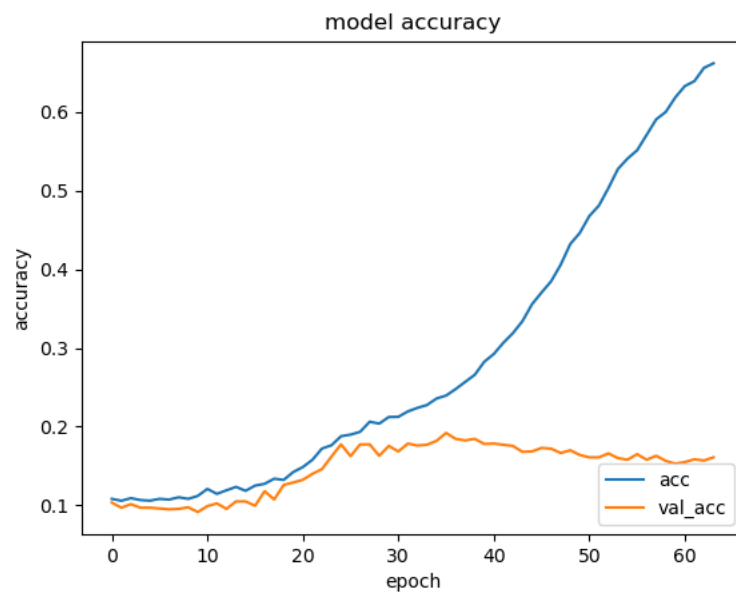


図 6.2.3 $N=26$, データ数 10000 MLP の実験結果

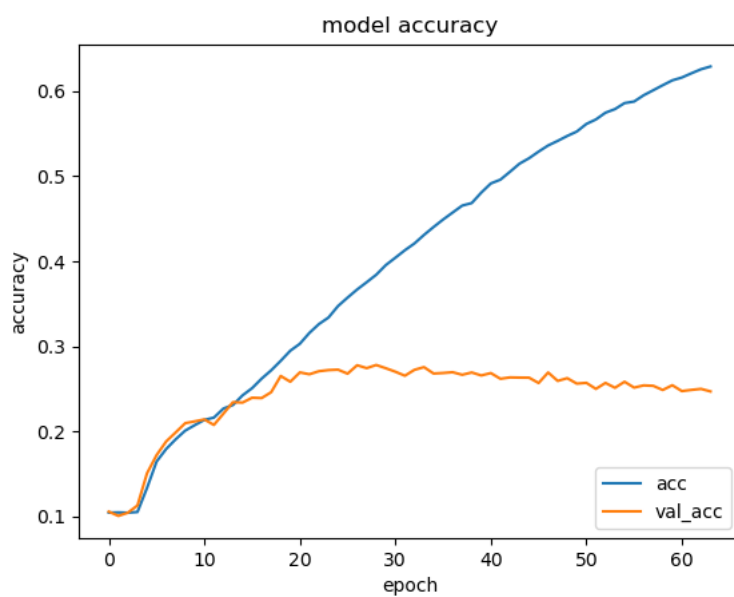


図 6.2.4 N=26, データ数 50000 MLP の実験結果

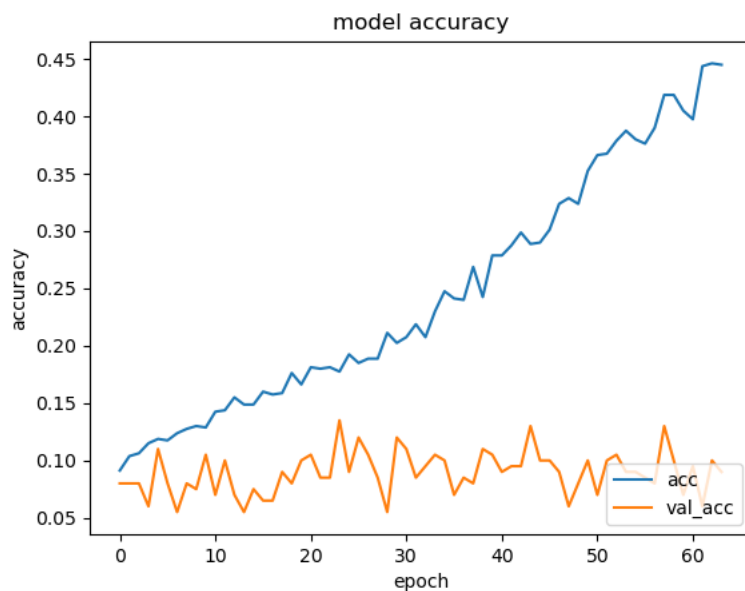


図 6.2.5 N=50, データ数 1000 MLP の実験結果

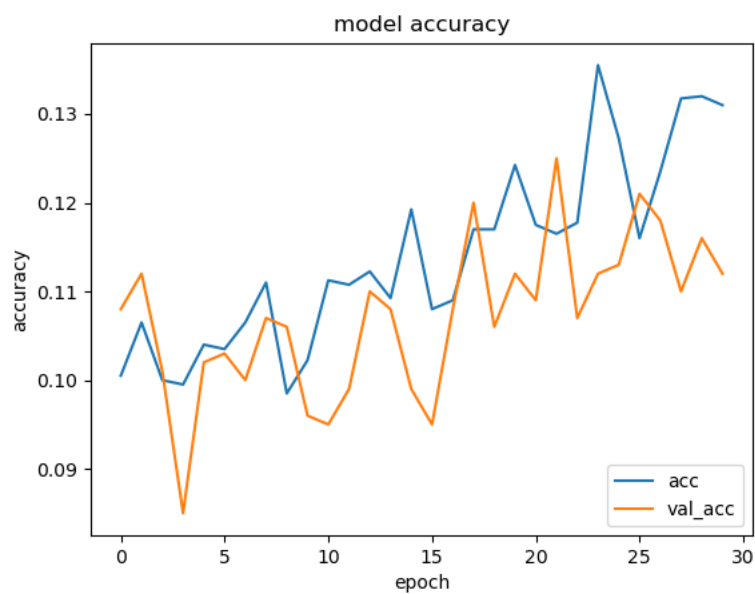


図 6.2.6 $N=50$, データ数 5000 MLP の実験結果

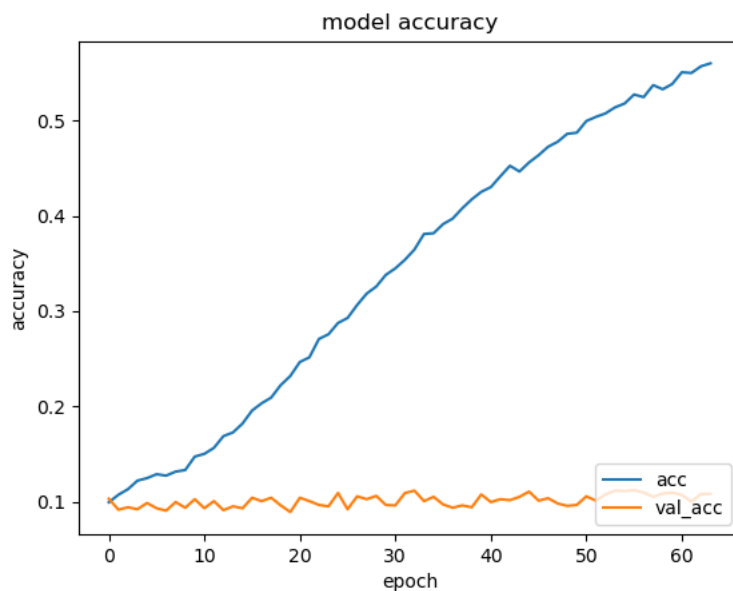


図 6.2.7 $N=50$, データ数 10000 MLP の実験結果

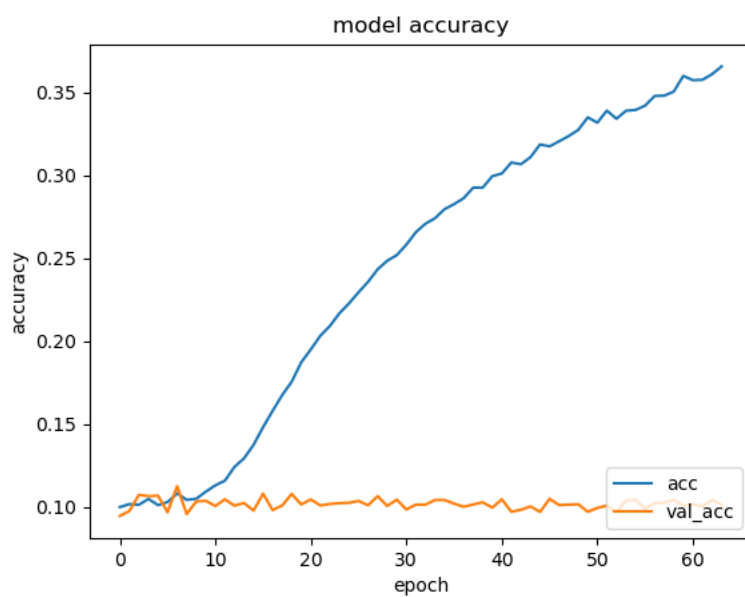


図 6.2.8 $N=50$, データ数 50000 MLP の実験結果

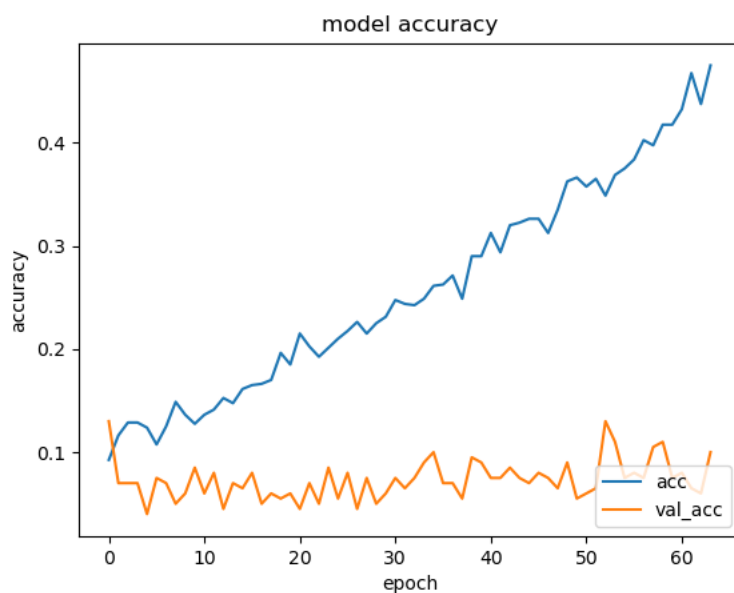


図 6.2.9 $N=100$, データ数 1000 MLP の実験結果

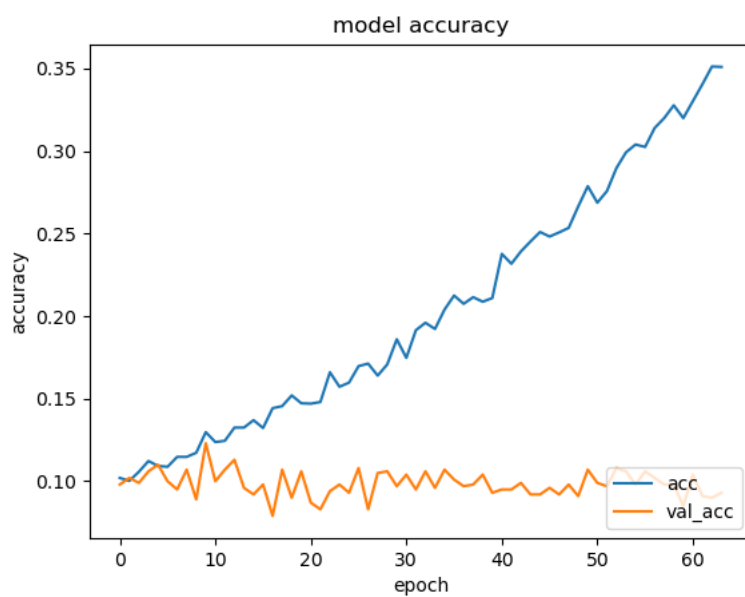


図 6.2.10 $N=100$, データ数 5000 MLP の実験結果

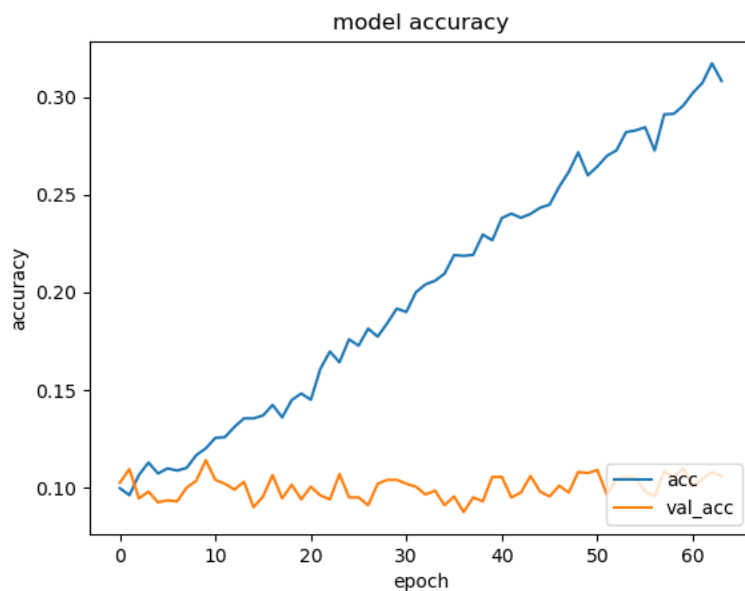


図 6.2.11 $N=100$, データ数 10000 MLP の実験結果

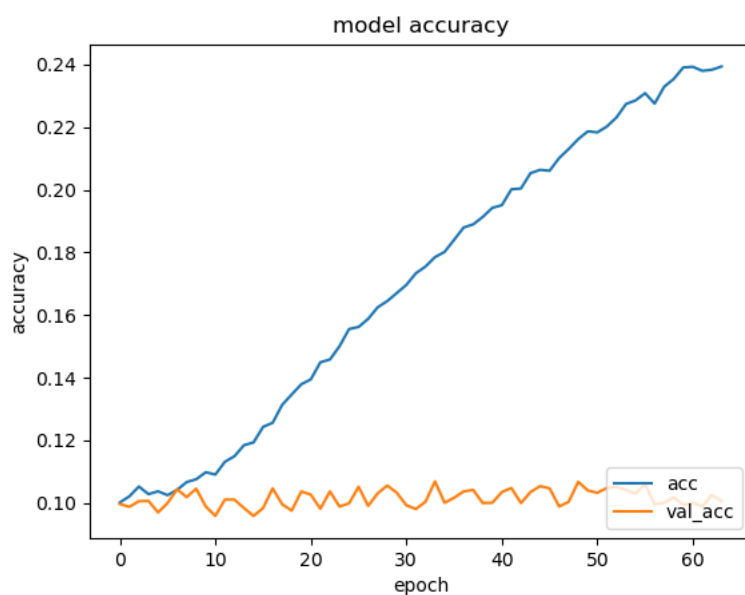


図 6.2.12 N=100, データ数 50000 MLP の実験結果

6.3 埋め込み層と長短期記憶を用いた実験の結果

ユーザーが覚える必要がある画像の枚数 n について、 $n=26, 30, 50, 100$ に対して実験を行った。ここで、実験に用いた機械学習モデルは表 6.3.1 のようなものである。

層 id	層	ノード数	出力形式(行列やテンソルの形式)

1	入力層		(14)
2	埋め込み層	14(語彙数)	(14, 3)
4	LSTM	56	(56)
5	LSTM	56	(56)
6	LSTM	56	(56)
7	全結合	50	(50)
8	全結合	50	(50)
9	全結合	50	(50)
10	パーセプトロン(出力層)	10	(10)

表 6.3.1 埋め込みと長短期記憶の構成

このモデルを訓練した結果、次の表 6.3.2 の結果となった。

データ数	N=26	N=50	N=100
1000	10.20%	17.20%	14.20%
5000	16.24%	10.51%	10.02%
10000	55.81%	10.23%	11.21%
50000	33.02%	11.09%	10.21%

表 6.3.2 埋め込みと長短期記憶による正解率

学習が最も成功したケースでは正解率は 55%程度まで向上した。また、データ数の多寡にかかわらず正解率が変化する傾向が見られるが、これは背景知識で説明したように局所解のトラップ現象によるものであると考えられる。そのため、現実的な攻撃者は複数回機械学習モデルを訓練し、最良のモデルで攻撃が行われることが考えられる。

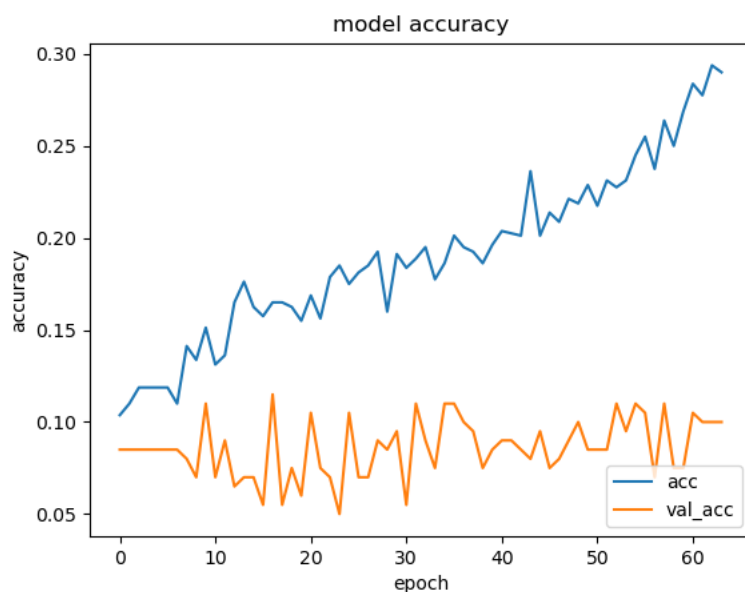


図 6.3.1 $N=26$, データ数 1000 LSTM の実験結果

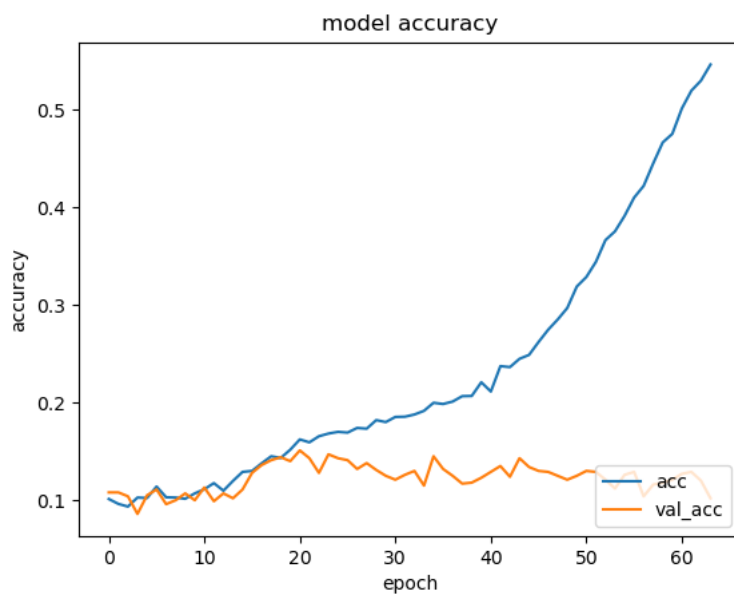


図 6.3.2 N=26, データ数 5000 LSTM の実験結果

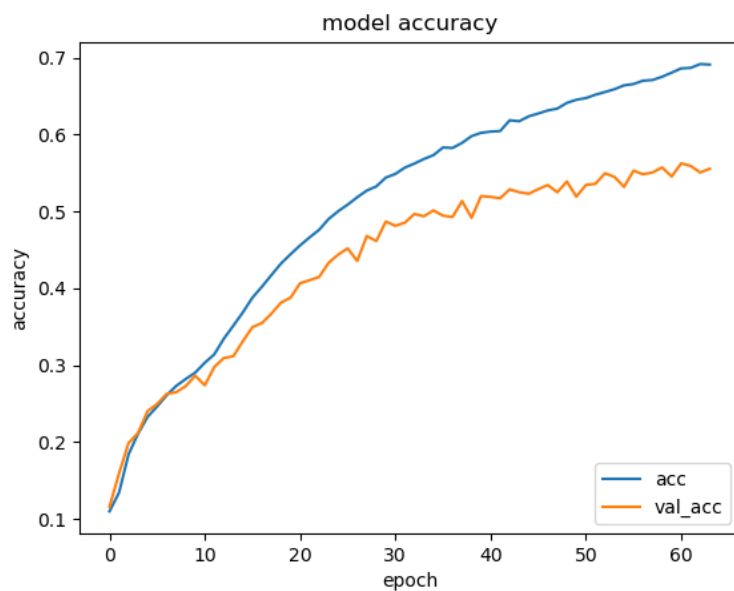


図 6.3.3 N=26, データ数 10000 LSTM の実験結果

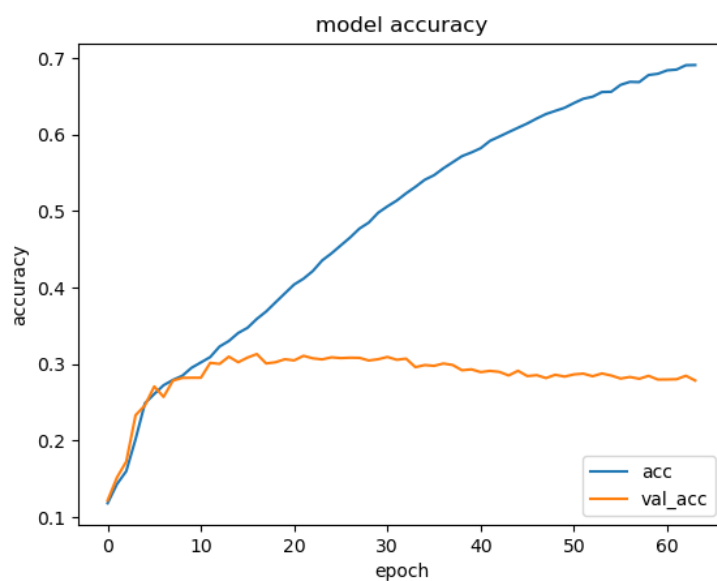


図 6.3.4 $N=26$, データ数 50000 LSTM の実験結果

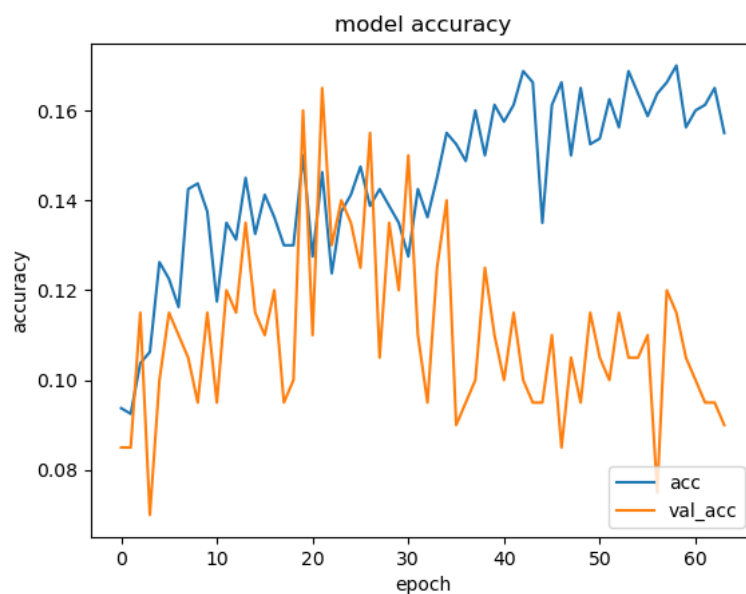


図 6.3.5 $N=50$, データ数 1000 LSTM の実験結果

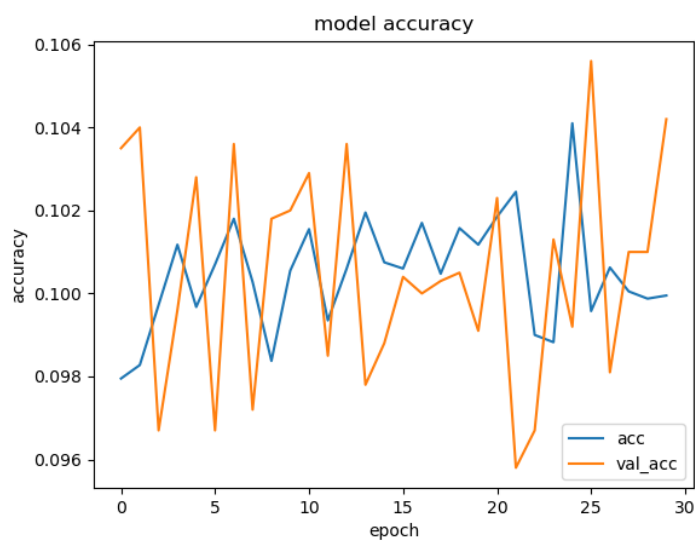


図 6.3.6 $N=50$, データ数 5000 LSTM の実験結果

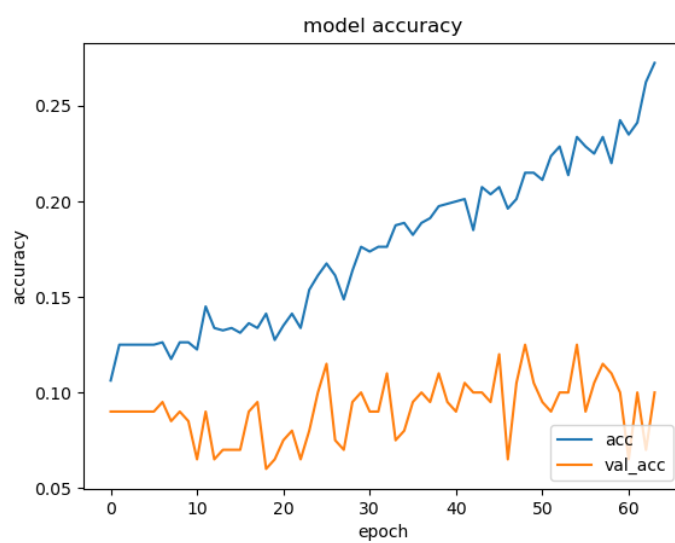


図 6.3.7 $N=50$, データ数 10000 LSTM の実験結果

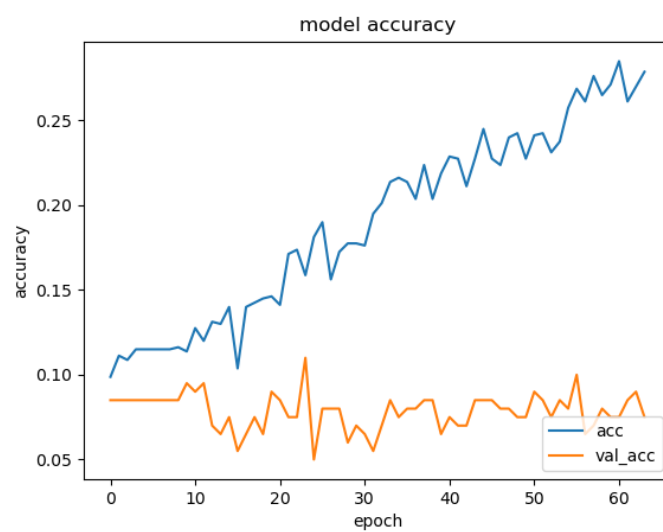


図 6.3.8 $N=50$, データ数 50000 LSTM の実験結果

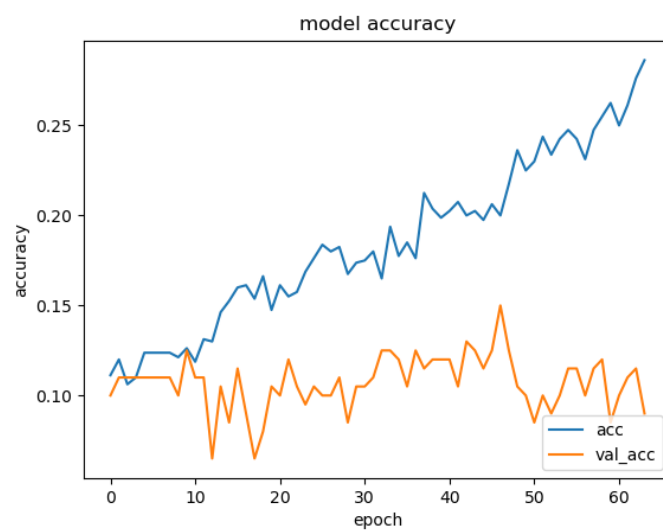


図 6.3.9 $N=100$, データ数 1000 LSTM の実験結果

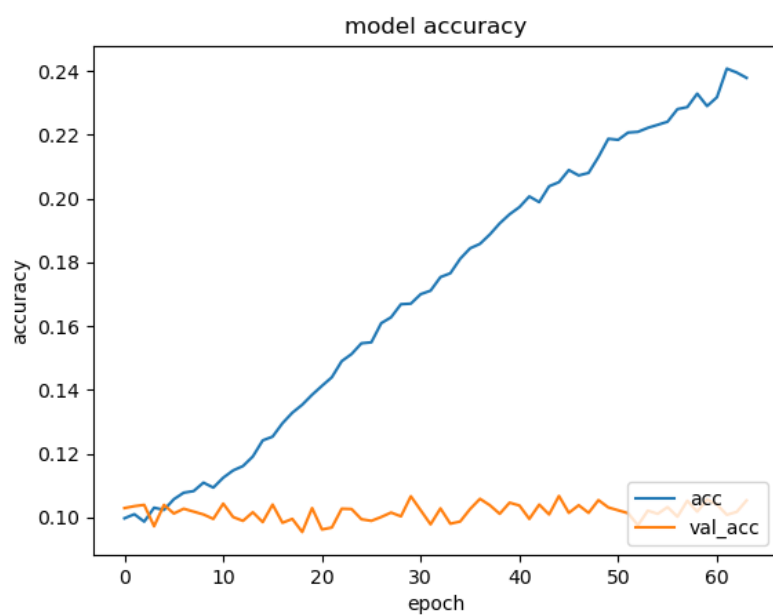


図 6.3.10 $N=100$, データ数 5000 LSTM の実験結果

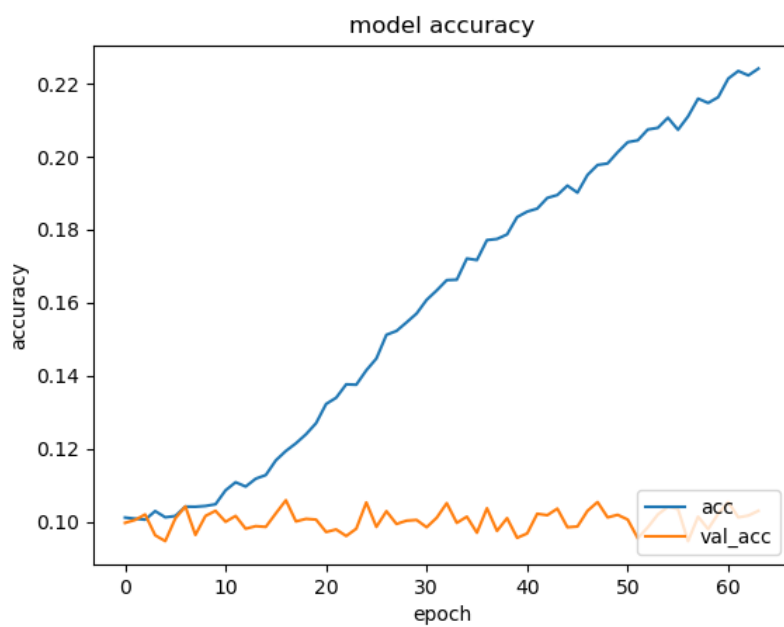


図 6.3.11 $N=100$, データ数 10000 LSTM の実験結果

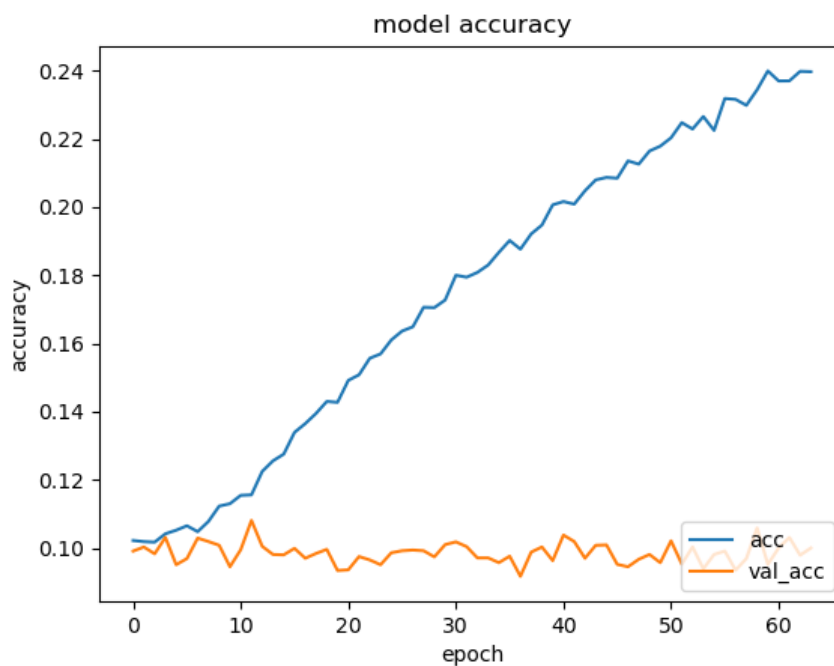


図 6.3.12 N=100, データ数 50000 LSTM の実験結果

6.4 考察

今回の実験から、N=26 程度の場合は正解率は最大 55%程度まで向上することがわかった。また、正解率向上は確率的なものである可能性が高いことが推測される。

今回の実験により、 σ 関数を埋め込み層が近似的に表現できている可能性が高いことが推察される。

また、ユーザーの記憶する画像の枚数が増加するにつれて突破が難しくなることがわかった。これは入力データのばらつきが大きくなることが原因として考えられ

る。

7 結論

今回の実験を通して、Manuel Blum らによって提案された人間計算可能なパスワードの安全性を検証することができた。ユーザーがたかだか 26 枚程度の画像を利用して認証を行う場合は 40%程度の確率で突破されることに対して、 $N=30$, $N=50$, ... と増加するにつれて認証の安全性が高まることが確認された。よって、人間計算可能なパスワードを安全に利用するためには、画像枚数を増加させる必要があることがわかる。

ただし、今回試したモデルはほんの一部であるため、他のモデルを活用すれば画像数 100 の場合でも突破できる可能性は否定できない。また、Preferred Networks 社の optuna など、機械学習モデルの自動探索を行うツールも存在するため、現実の攻撃者は労力をかけずにモデルの探索を行うことが予測される。これらの可能性を考えると、人間計算可能なパスワードの安全性の検証は今後とも必要であると考えられる。

本研究は一つのテーマであった「機械学習モデルを用いたセキュリティ手法自体の評価」も行うことができた。人間計算可能なパスワードを例に、安全に使える条件の目安を示すことができたと考えている。一方で、機械学習を用いた安全性評価の限界を示すこともできたと考えており、今後機械学習を用いてセキュリティ研究を行う上での懸念点も示せたのではないかと考えている。

謝辞

本研究を進めるにあたり、ご多忙にも関わらず熱心にご指導いただきました櫻井幸一教授に心から感謝いたします。また、研究にご協力いただき、多数の助言をいただきました櫻井研究室の皆様、特に村田壱生先輩に心から感謝いたします。

参考文献

- [1] Manuel Blum, Santosh Vempala. The complexity of human computation via a concrete model with an application to passwords. Proceedings of the National Academy of Sciences, July 2017.
- [2] Jeremiah Blocki, Manuel Blum, Anupam Datta, Santosh Vempala. Towards Human Computable Passwords. Innovations in Theoretical Computer Science (ITCS). January 2017
- [3] 村田壱生, 人間計算可能なパスワード認証に対する深層学習による安全性評価, 九州大学理学部物理学情報理学コース卒業論文, 2022
- [4] 丸野美矩人, 人間計算可能なパスワードに対する長短期記憶ニューラルネットワークによる安全性評価, 九州大学工学部電気情報工学科卒業論文, 2023
- [5] 斎藤 康毅, ゼロから作る Deep Learning —Python で学ぶディープラーニングの理論と実装, オライリー, 2016
- [6] 斎藤 康毅, ゼロから作る Deep Learning 2 —自然言語処理編, オライリー, 2018
- [7] Antonio Gulli (著), Sujit Pal (著), 大串 正矢 (翻訳), 久保 隆宏 (翻訳), 中山 光樹 (翻訳), 直感 Deep Learning —Python×Keras でアイデアを形にするレシピ, オライリー, 2018
- [8] Chiheb Chebbi (著), 新井 悠 (翻訳), 一瀬 小夜 (翻訳), 黒米 祐馬 (翻訳), セキュリティエンジニアのための機械学習 —AI 技術によるサイバーセキュリティ対策入門, オライリー, 2021
- [9] Okta, SAML ガイド, <https://support.okta.com/help/s/article/Beginners-Guide-to-SAML?language=ja>
- [10] Matlab, 深層学習ニューラル ネットワーク用の単語埋め込み層, <https://jp.mathworks.com/help/textanalytics/ref/nnet.cnn.layer.wordembeddinglayer.html>
- [11] Yushuo Guan, Yuanxing Zhang, Lin Chen, Kaigui Bian. A Neural Attack Model for Cracking Passwords in Adversarial Environments. IEEE International Conference on Communications in China (ICCC). 2019. 183-188
- [12] Manuel Blum, Santosh Vempala. The complexity of human computation via a concrete model with an application to passwords. Proceedings of the National Academy of Sciences of the USA, Volume117 (PNAS). 2020. 9208-9215
- [13] Elan Rosenfeld, Santosh Vempala, Manuel Blum. Human-Usable Password Schemas: Beyond Information-Theoretic Security. Computing Research Repository, June 2019.
- [14] 今泉允聡. 深層学習の原理に迫る数学の挑戦. 岩波書店. 2021
- [15] CYBERNET. シングルサインオン, <https://www.cybernet.co.jp/onelogin/function/saml.html>, 2021
- [16] トラストログイン, SSO とは, <https://trustlogin.com/sso/> 2021
- [17] トレンドマイクロ株式会社, パスワードの利用実態調査, <https://www.trendmicro.com/ja-jp/about/press-release/2020/pr-20200929-01.html> 2020
- [18] PASONA, ディープラーニングとは,

<https://www.pasonatech.co.jp/workstyle/column/detail.html?p=2181>, 2022

[19] Mathworks, ディープラーニングとは？これだけは知っておきたい特徴,

<https://jp.mathworks.com/discovery/deep-learning.html>

[20] AUC, RNN と LSTM の違いと特徴,

<https://www.acceluniverse.com/blog/developers/2019/07/lstm.html>

[21] aismiley, 自然言語処理に使われる LSTM とは？, <https://aismiley.co.jp/ai-news/lstm/>

[22] AISIA, LSTM Long Short Term Memory, <https://products.sint.co.jp/aisia/blog/vol1-18>

[23] FIDO, パスキー認証, <https://fidoalliance.org/passkeys/?lang=ja>

[24] OneLogin, OTP, HOTP, TOTP の違い, <https://www.onelogin.com/jp-ja/learn/otp-totp-hotp>

[25] Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2014

[26] Keras-team, Keras, <https://github.com/keras-team/keras>

[27] Keras, Keras API Documentation, <https://keras.io/>

[28] Tensorflow, Tensorflow Documentation, <https://www.tensorflow.org/?hl=ja>

[29] Tensorflow, Tensorflow Docs, <https://github.com/tensorflow/docs>

[30] Tensorflow, An Open Machine Learning Framework for Everyone, <https://github.com/tensorflow/tensorflow>

学会発表

池田悠登, 櫻井幸一, 人間計算可能なパスワード認証に対する 埋め込み型の双方向 LSTM を用いた安全性実験評価, 情報処理学会 第 198 回マルチメディア通信と分散処理・第 104 回コンピュータセキュリティ合同研究発表会 2024 年 3 月 18 日(月)-19 日(火) 発表予定

索引

CSPSolver.....	iv, vii, 16, 17	人間計算可能なパスワード ...i, ii, iv, 1, 2, 14, 15, 19, 20, 35, 36, 40
One-Hot 表現.....	8	
チャレンジ - レスポンス.....	ii	人間計算可能な定義.....iv, 14
ユーザー認証.....	iv, 1, 2, 5	生体認証.....iv, 2, 3, 5
一桁チャレンジ.....	iv, vi, 15, 16, 20	多層パーセプトロン .ii, iv, vi, vii, 1, 2, 7, 8, 9, 10, 17, 18, 19, 21
過学習.....	8, 21	知識認証.....1, 2, 3, 6, 14
機械学習 ii, iv, 1, 2, 7, 8, 9, 10, 11, 13, 18, 19, 20, 21, 27, 28, 35, 36, 38		長短期記憶i, ii, iv, v, vii, 12, 18, 19, 27, 28, 36
逆誤差伝搬法.....	10, 11	埋め込み .. i, ii, iv, v, vi, vii, 11, 12, 19, 20, 21, 27, 28, 35, 36
最適化アルゴリズム.....	8, 10, 11, 13	
所有物認証.....	2, 3, 5, 6	

付録: プログラムコード

本プログラムは効率的な実験の進行と各種パスワード方式の管理、機械学習モデルの管理を目的として python を用いて複数のファイルを用いて記述を行った。各ファイルのプログラムコードを添付してある。

```
batch_run.py
from computable_password_generator import ComputablePasswordGenerator
from utils import Utils
from models import Models
import os
import traceback

iter = 0
try:
    os.mkdir("outputs/{}".format(iter))
except:
    pass
for model in Models.list_models():
    print("Runnnng model: {}".format(model.name))
    for generator in ComputablePasswordGenerator.list_generators():
        try:
            print("Testing: generator: {}, model: {}".format(generator.name, model.name))
            print("Figure name: {}".format(generator.name + "_" + model.name))
            generated_passwords = generator.generator(model.required_data_size)
            x_train, x_test, y_train, y_test = Utils.split_to_train_and_valid(generated_passwords)
            x_train = model.resharper(x_train)
            x_test = model.resharper(x_test)
            model.model.fit(x_train,y_train,batch_size=model.batch_size, epochs=model.epochs,
verbose=1, validation_data=(x_test, y_test))
            history = model.model.history.history
            Utils.plot_history(history, "{}"/.format(iter) + generator.name + "_" + model.name)
        except Exception as e:
            print("Error: generator: {}, model: {}".format(generator.name, model.name))
            print(e)
            traceback.print_exc()
            continue
```

```

utils.py
from matplotlib import pyplot
from tensorflow import keras
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

class Utils:
    # グラフのプロットを行う関数
    # keras.model.fit.history および画像の保存名を引数として取る
    @staticmethod
    def plot_history(history :str, name :str) -> None:
        # 学習時の学習データおよび正解データに対する正解率を表示する
        pyplot.clf()
        pyplot.plot(history['accuracy'])
        pyplot.plot(history['val_accuracy'])
        pyplot.title('model accuracy')
        pyplot.xlabel('epoch')
        pyplot.ylabel('accuracy')
        pyplot.legend(['acc', 'val_acc'], loc='lower right')
        pyplot.savefig("outputs/" + name + '_accuracy.png')

        # 学習時の学習データおよび正解データに対する損失関数の値を表示する
        pyplot.clf()
        pyplot.plot(history['loss'])
        pyplot.plot(history['val_loss'])
        pyplot.title('model loss')
        pyplot.xlabel('epoch')
        pyplot.ylabel('loss')
        pyplot.legend(['loss', 'val_loss'], loc='lower right')
        pyplot.savefig("outputs/" + name + '_loss.png')
        return None

    @staticmethod
    def split_to_train_and_valid(generated_passwords :pd.DataFrame) -> (np.ndarray, np.ndarray,
np.ndarray, np.ndarray):

```

```

generated_passwords = generated_passwords.sample(frac=1).reset_index(drop=True)
x = generated_passwords.drop(labels = ["Z"],axis = 1)
y = generated_passwords["Z"]

# 説明変数・目的変数をそれぞれ訓練データ・テストデータに分割
x_train,x_test,y_train,y_test = train_test_split(x, y, test_size=0.2)

# データの型変換
#x_train = x_train.astype(float)
#x_test = x_test.astype(float)

y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)

return x_train, x_test, y_train, y_test

class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs={}):
        self.losses = []

    def on_batch_end(self, batch, logs={}):
        self.losses.append(logs.get('loss'))

computable_password_generator.py
import numpy as np
import pandas as pd

class ComputablePasswordGenerator:
    # 人間計算可能なパスワードの流出データを模したデータを自動生成する関数群
    # このクラスの関数を実行すると、人間計算可能なパスワードを模したデータが生成され、
    csv ファイルとして保存される
    # 外部のプログラムから呼び出すときは、from computable_password_generator import
    ComputablePasswordGenerator として import を行う

    # この関数群の呼び出しには、int: データ数を引数として与える
    # この関数群は pandas.DataFrame を return する
    class Utils:

```

```

    @staticmethod
    def sgm(n :int) -> np.ndarray:
        sgm = np.random.randint(0,10,n)
        return sgm

class GeneratorWithMetadata:
    def __init__(self, generator, name :str):
        self.generator = generator
        self.name = name

    @staticmethod
    def list_generators() -> list:
        generators = []

generators.append(ComputablePasswordGenerator.GeneratorWithMetadata(ComputablePasswordGenerator.s_x, "s_x"))

generators.append(ComputablePasswordGenerator.GeneratorWithMetadata(ComputablePasswordGenerator.password_with_middle, "middle"))

#generators.append(ComputablePasswordGenerator.GeneratorWithMetadata(ComputablePasswordGenerator.password_simple_add, "simple_add"))

return generators

    @staticmethod
    def password_simple_add(datasize :int) -> np.ndarray:
        result = []
        for row in range(datasize):
            X = ComputablePasswordGenerator.Utils.sgm(14)
            Z = (X[0] + X[1] + X[2]) % 10
            row = np.append(X, Z)
            result.append(row)
        table_array = np.array(result)
        return pd.DataFrame(table_array, columns=["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
            "X8", "X9", "X10", "X11", "X12", "X13", "Z"])

    @staticmethod

```



```

def password_with_middle(datasize :int) -> np.ndarray:
    result = []
    sgm = ComputablePasswordGenerator.Utls.sgm(14)
    for row in range(datasize):
        X = ComputablePasswordGenerator.Utls.sgm(14)
        S_X = np.zeros(14)
        for k in range(14):
            S_X[k] = sgm[X[k]]
        mid = (S_X[10] + S_X[11]) % 10
        Z = (S_X[int(mid)] + S_X[12]) % 10
        row = np.append(X, Z)
        result.append(row)
    table_array = np.array(result)
    return pd.DataFrame(table_array, columns=["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
        "X8", "X9", "X10", "X11", "X12", "X13", "Z"])

```

@staticmethod

```

def s_x(datasize :int) -> np.ndarray:
    result = []
    sgm = ComputablePasswordGenerator.Utls.sgm(14)
    for row in range(datasize):
        X = ComputablePasswordGenerator.Utls.sgm(14)
        S_X = np.zeros(14)
        for k in range(14):
            S_X[k] = sgm[X[k]]
        mid = ( S_X[10] + S_X[11] ) % 10
        Z = ( S_X[12] + S_X[13] + S_X[int(mid)] ) % 10
        row = np.append(X, Z)
        result.append(row)
    table_array = np.array(result)
    return pd.DataFrame(table_array, columns=["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
        "X8", "X9", "X10", "X11", "X12", "X13", "Z"])

```

Models.py

```

from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Dropout, LSTM, Flatten, Bidirectional, Concatenate,
Input, Embedding, Reshape

```

```

import pandas as pd
import numpy as np
import tensorflow as tf

class Models:
    class ModelWithMetadata:
        def __init__(self, model, name :str, batch_size :int, epochs :int, required_data_size :int):
            self.model = model
            self.name = name
            self.batch_size = batch_size
            self.epochs = epochs
            self.required_data_size = required_data_size

        def resharper(self, df :pd.DataFrame) -> (np.ndarray):
            if self.name.find("lstm") != -1:
                return df.to_numpy().reshape(df.shape[0], df.shape[1], 1)
            return df

    @staticmethod
    def list_models() -> list:
        models = []
        models.append(Models.ModelWithMetadata(Models.embed_mlp(), "mlp_with_embedding",
25, 1024, 50000))
        models.append(Models.ModelWithMetadata(Models.embed_lstm(), "lstm_with_embedding",
25, 200, 50000))
        return models

    @staticmethod
    def embed_mlp() -> Model:
        inputs = Input(shape=(14, 1))
        inputs_resaped = Flatten()(inputs)
        embedding = Embedding(14, 3)(inputs)
        embedding = Flatten()(embedding)
        concat = Concatenate(axis = 1)([inputs_resaped, embedding])
        dense = Dense(50)(concat)
        dense = Dense(50)(dense)

```

```

dense = Dense(50)(dense)
outputs = Dense(10, activation="softmax")(dense)
model = Model(inputs=inputs, outputs=outputs)
model.compile(loss="mean_squared_error", optimizer="Adam", metrics=["accuracy"])
model.summary()
return model

@staticmethod
def embed_lstm() -> Model:
    inputs = Input(shape=(14, 1))
    inputs_reshaped = Flatten()(inputs)
    embedding = Embedding(14, 1)(inputs)
    embedding = Flatten()(embedding)
    embedding = tf.squeeze(embedding, axis=2)
    lstm = Bidirectional(LSTM(56), return_sequences=True)
    lstm = Bidirectional(LSTM(56), return_sequences=True)
    lstm = Bidirectional(LSTM(56), return_sequences=True)
    dense = Dense(50)(lstm)
    dense = Dense(50)(lstm)
    dense = Dense(50)(lstm)
    outputs = Dense(10, activation="softmax")(dense)
    model = Model(inputs=inputs, outputs=outputs)
    model.compile(loss="mean_squared_error", optimizer="Adam", metrics=["accuracy"])
    model.summary()
    return model

```